

Pipelined CPU Design

Slides adapted from Ashmita Sinha (CPU Architect, Intel Corporation)

Quick survey for one of next week's lecture on floating-point adder and multiplier design. The speaker has a conflict. So, we need to find another time slot.



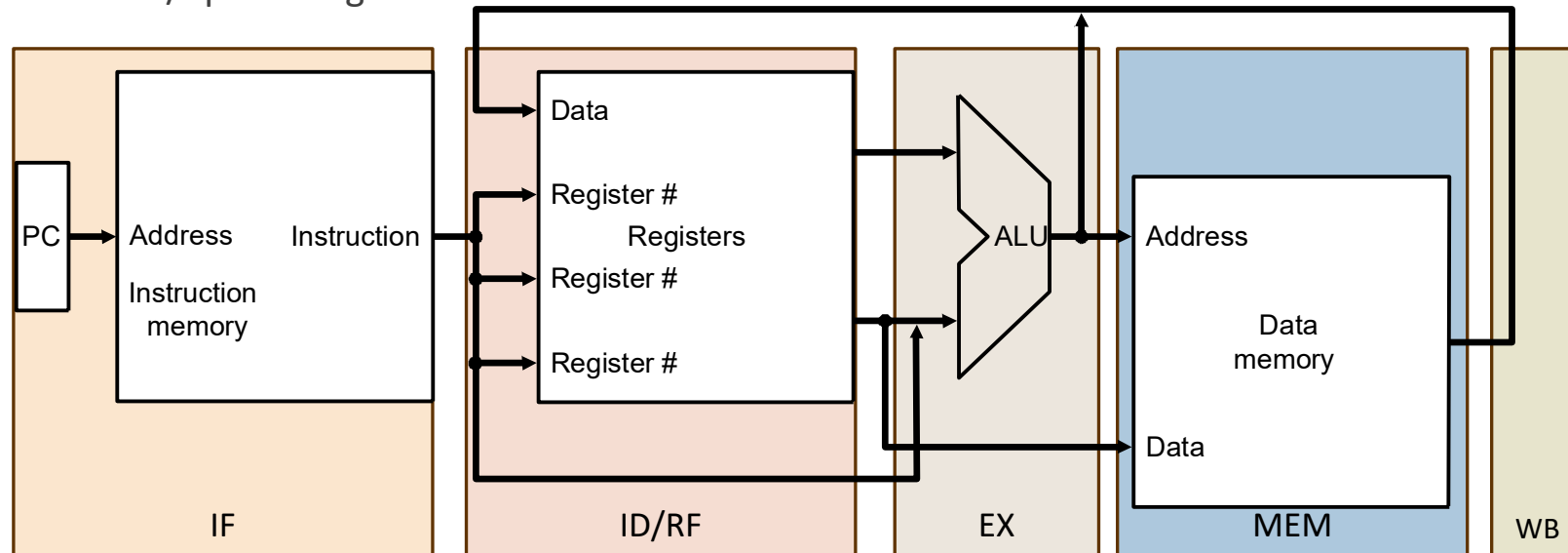
References/Credits

- Books references:
 - H&P
 - H&H, Digital Design & Computer Architecture, Chapter 7
 - [7---Microarchitecture--With-contributions-f 2013 Digital-Design-and-Computer.pdf](#)
- Prof Onur Mutlu's lecture on Pipelined CPUs, ETH Zurich, Spring'25
- Modern day CPU microarchitecture example
 - https://chipsandcheese.com/p/skylake-intels-longest-serving-architecture?utm_source=publication-search

Thanks to Ashmita Sinha (Intel) for putting the original version of this slide deck together.

MIPS Single Cycle : Recall

- IF : Instruction Fetch (instruction pointed by PC)
- ID/RF : Instruction Decode (ID) and fetching operands from register file (RF)
- EX : Execute instruction or evaluate effective memory address
- MEM : Memory Fetch
- WB : Write Back/update register file



MIPS Single Cycle : Recall

Machine Encoding

op		rs=base		rt		imm=offset	
lw (35)		base		rt		offset	
31	26	25	21	20	16	15	0

I-Type

[PC] : lw \$s3, 8(\$s0)

Fetch:

Instr <- INST_MEM[PC]
PC <- PC + 4

Decode:

Generate Control Signals (Instr)
RF : Read \$s0
Sign extend offset (8)

Execute:

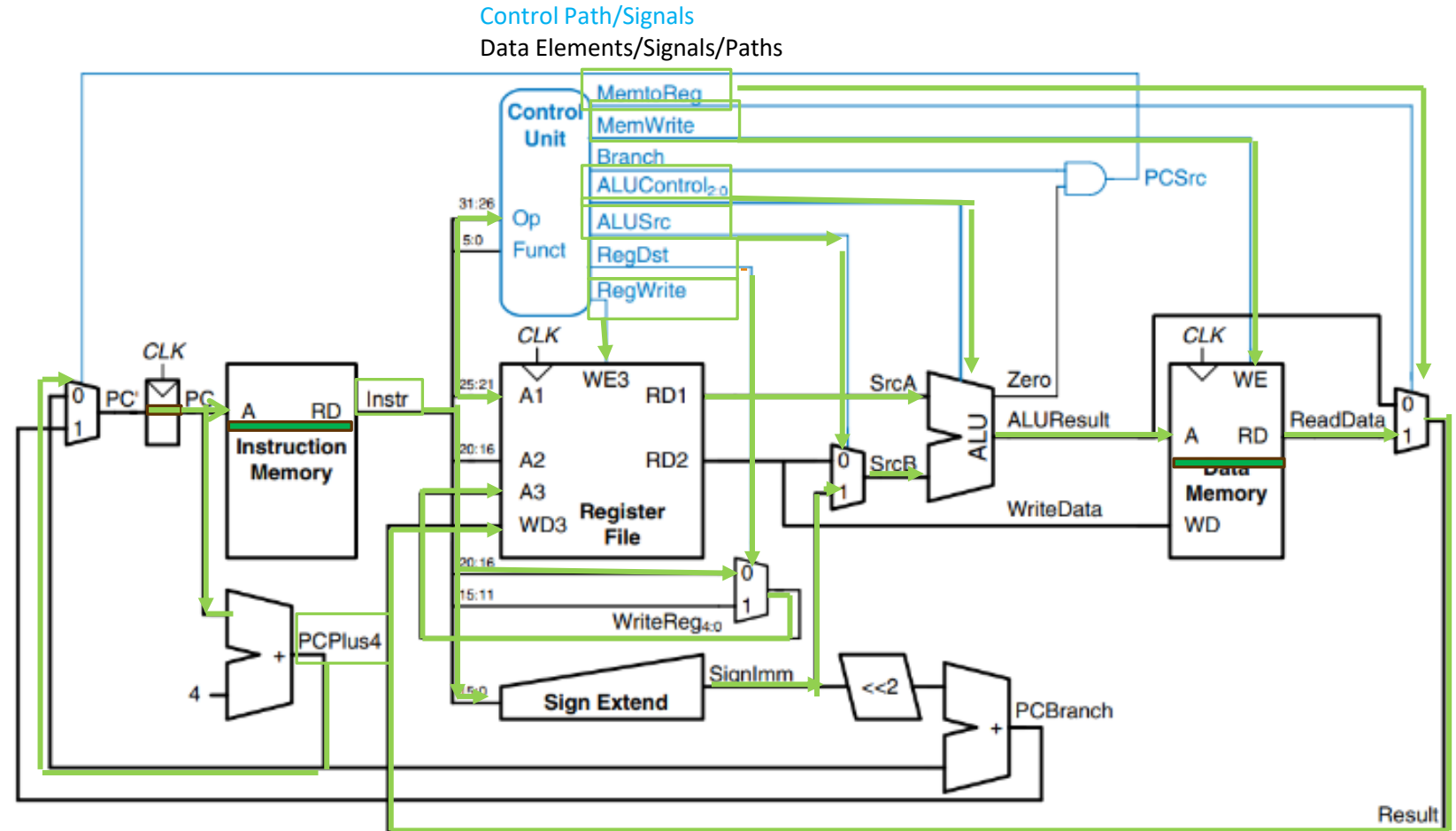
Compute effective address
Addr <- sign_ext(8) + base (\$s0)

Data Memory:

ReadData <- DATA_MEM[addr]

Write Back:

Dest (\$s3) <- ReadData



MIPS Single Cycle : Really?

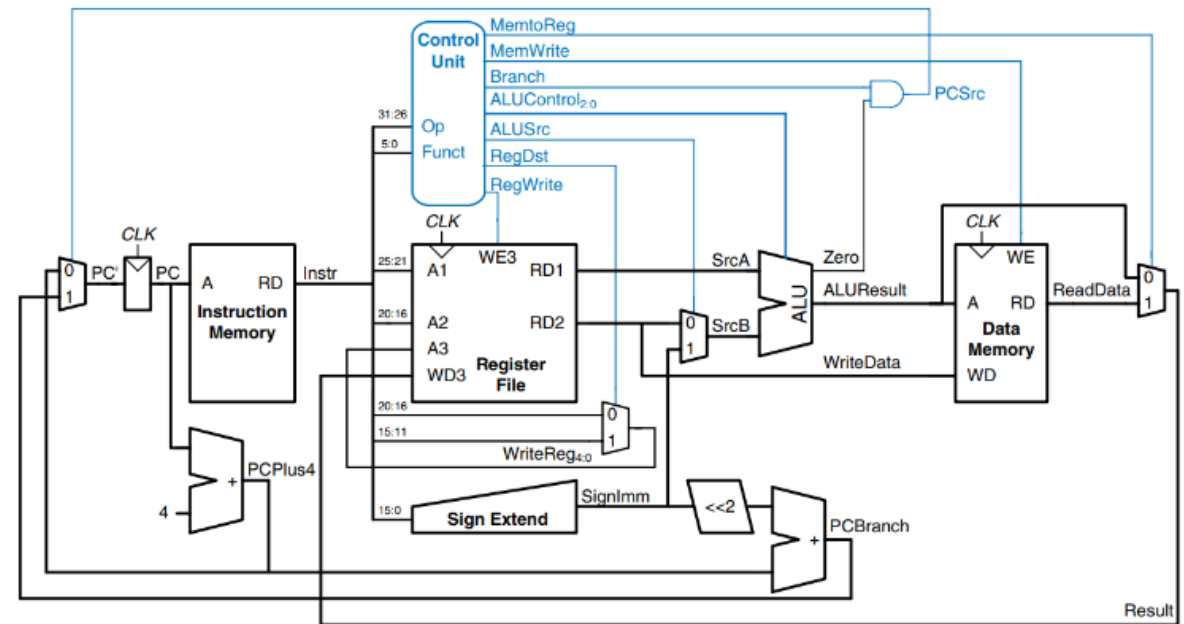
The register file and data memory are shown as clocked. Does that mean it takes 3 cycles?

No.

Instruction memory -> Read only. Not clocked.

Register file -> Read not clocked. Write clocked.

Data memory -> Read not clocked. Write clocked.



MIPS Single Cycle : Really?

During one clock cycle

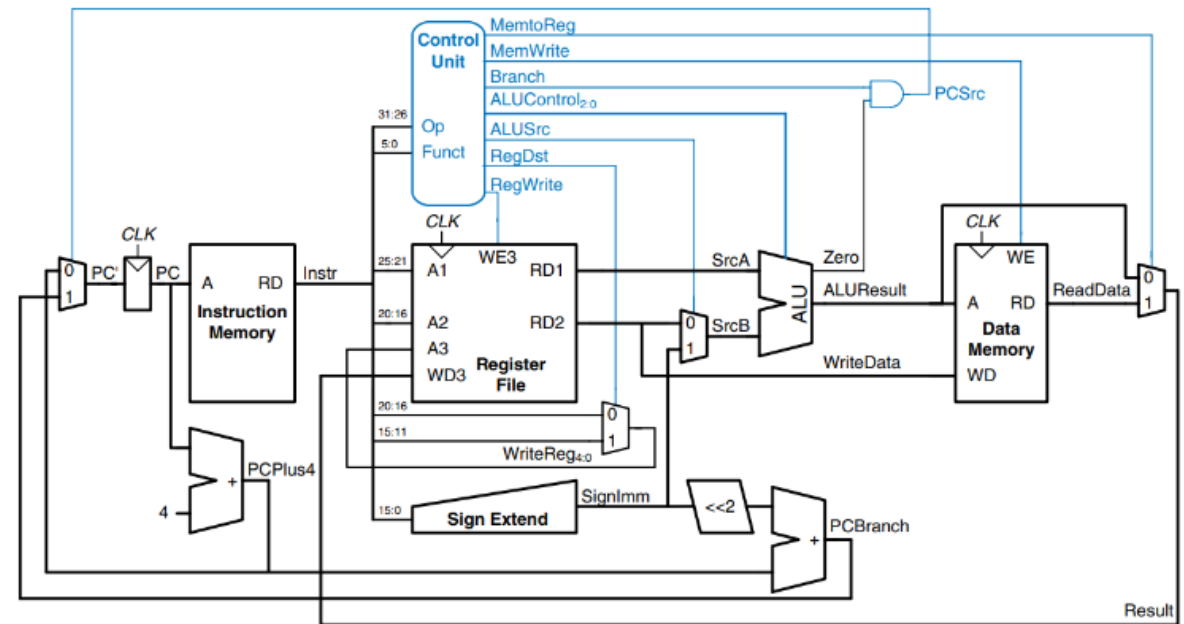
Suppose the clock edge occurs at time $t = 0$.

Immediately after the edge

The PC holds the address of the current instruction.
The register file contains the results written by the previous instruction (if any).

Throughout the rest of the cycle

Instruction memory outputs the instruction.
Register file reads RD1 and RD2.
The ALU computes.
Data memory is read (for a load).
Multiplexers choose the correct result.



MIPS Single Cycle : Really?

At the next clock edge

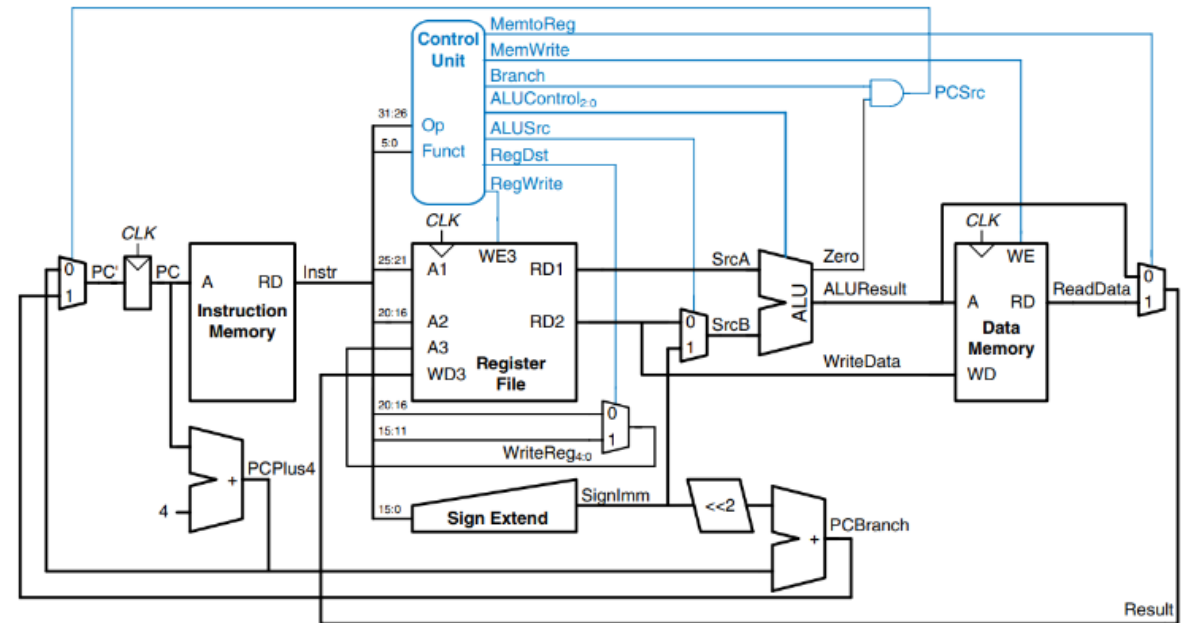
If RegWrite is asserted, the register file stores the new value.

If MemWrite is asserted, data memory stores the new value.

The PC updates to PC+4 or the branch target.

Then the next instruction begins.

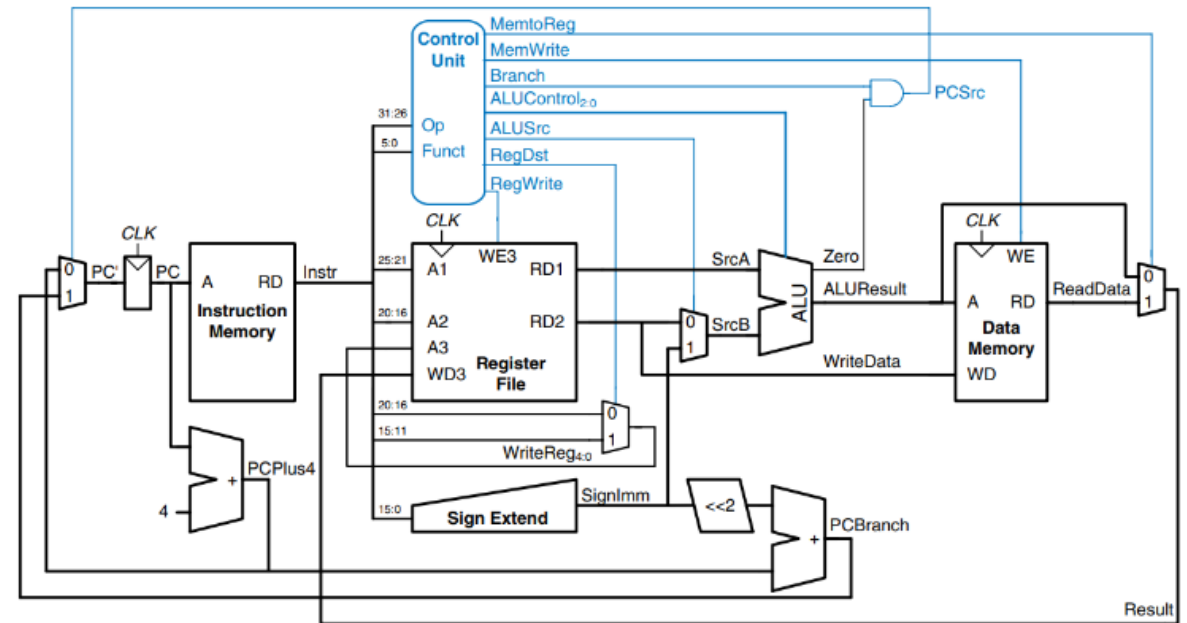
Everything between the two clock edges happens "within" that single cycle.



Single Cycle: What are the issues?

Area : Hardware utilization is not optimal

Power : Each data path element is always active (no clk gating possible)



Performance : Time Period (hence Clk Freq) limited by longest ISA instruction

CPU Performance Metrics

Throughput

Instructions per Cycle (IPC)

Latency

Cycles per Instruction (CPI)

Verilog Code for Single-Cycle Processor

- H&H, Digital Design & Computer Architecture, Chapter 7
 - [7---Microarchitecture--With-contributions-f_2013_Digital-Design-and-Computer.pdf](#)



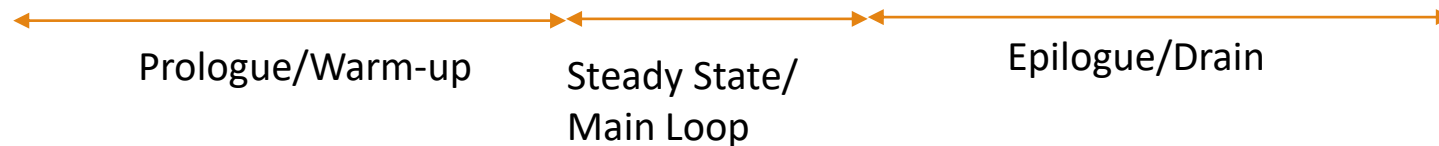
Pipelining – What is it?

Temporal parallelism

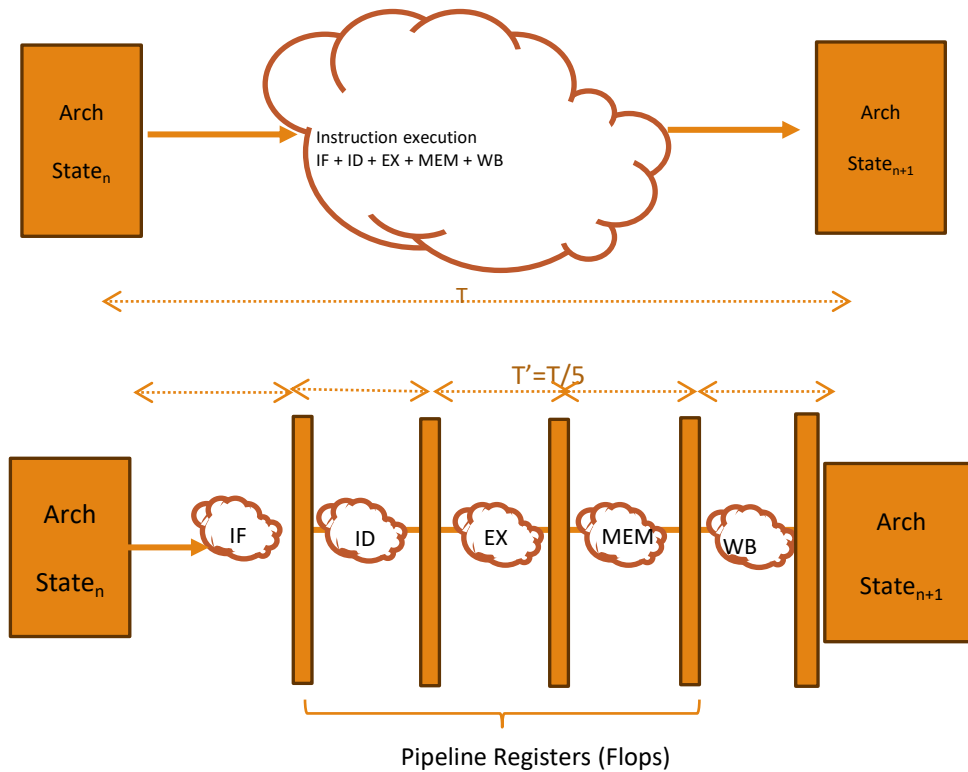
Each instruction broken up into series of steps

Multiple instructions execute at once

	cycle n	cycle n+1	cycle n+2	cycle n+3	cycle n+4	cycle n+5	cycle n+6	cycle n+7	cycle n+8	cycle n+9
Instr0	IF	ID	EX	MEM	WB					
Instr1		IF	ID	EX	MEM	WB				
Instr2			IF	ID	EX	MEM	WB			
Instr3				IF	ID	EX	MEM	WB		
Instr4					IF	ID	EX	MEM	WB	
Instr5						IF	ID	EX	MEM	WB



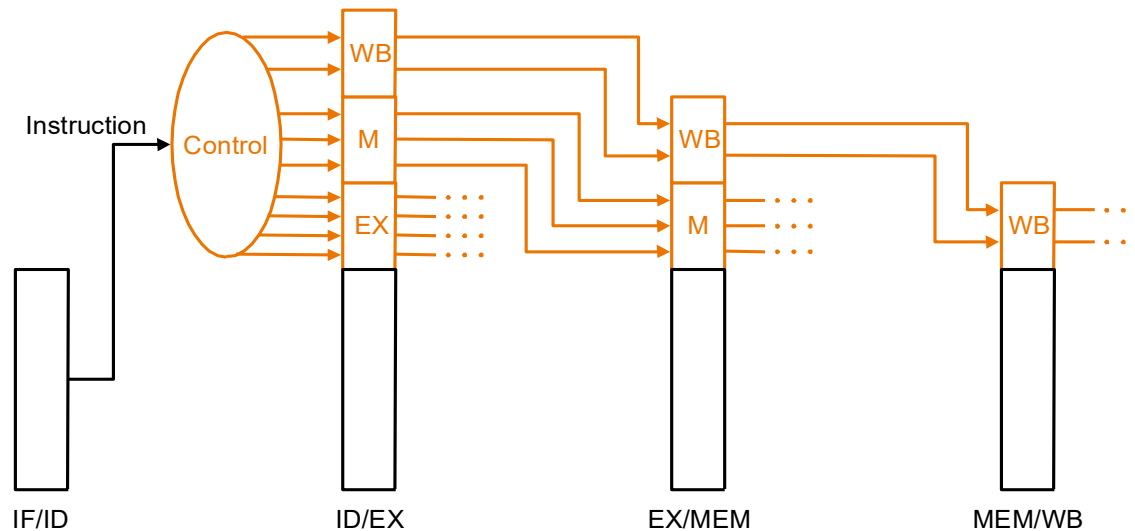
Pipelining : What is needed?



Pipelining Data Path

- Divide into stages
- Add pipeline registers to carry information along with each instruction
- Propagate data signals across stages, wherever required

Pipelining : What is needed?



Pipelining Control Path

- Propagate control signals to the right stages
- Carry instruction word in each stage, and generate control signals needed for each stage

Pipelining : Let's revisit CPU metrics

Performance:

- We are able to reduce clock time period, and hence frequency
- Cycle time is determined by slowest/longest pipeline stage
- Throughput has increased
- $Speedup = \frac{\text{Average instruction time without pipelining}}{\text{Average instruction time with pipelining}}$
 $= \frac{(100+50+100+100+50 \text{ ns})}{100 \text{ ns}} = 4 \text{ times}$

	cycle n	cycle n+1	cycle n+2	cycle n+3	cycle n+4	cycle n+5	cycle n+6	cycle n+7	cycle n+8	cycle n+9
Instr0	IF	ID	EX	MEM	WB					
Instr1		IF	ID	EX	MEM	WB				
Instr2			IF	ID	EX	MEM	WB			
Instr3				IF	ID	EX	MEM	WB		
Instr4					IF	ID	EX	MEM	WB	
Instr5						IF	ID	EX	MEM	WB

Power:

- We can gate using control signals if a given stage is not used (for instance MEM stage for ADD instruction)

100ns	50ns	100ns	100ns	50ns
IF	ID	EX	MEM	WB

Slowest pipe stage takes 100ns ⇔ cycle time

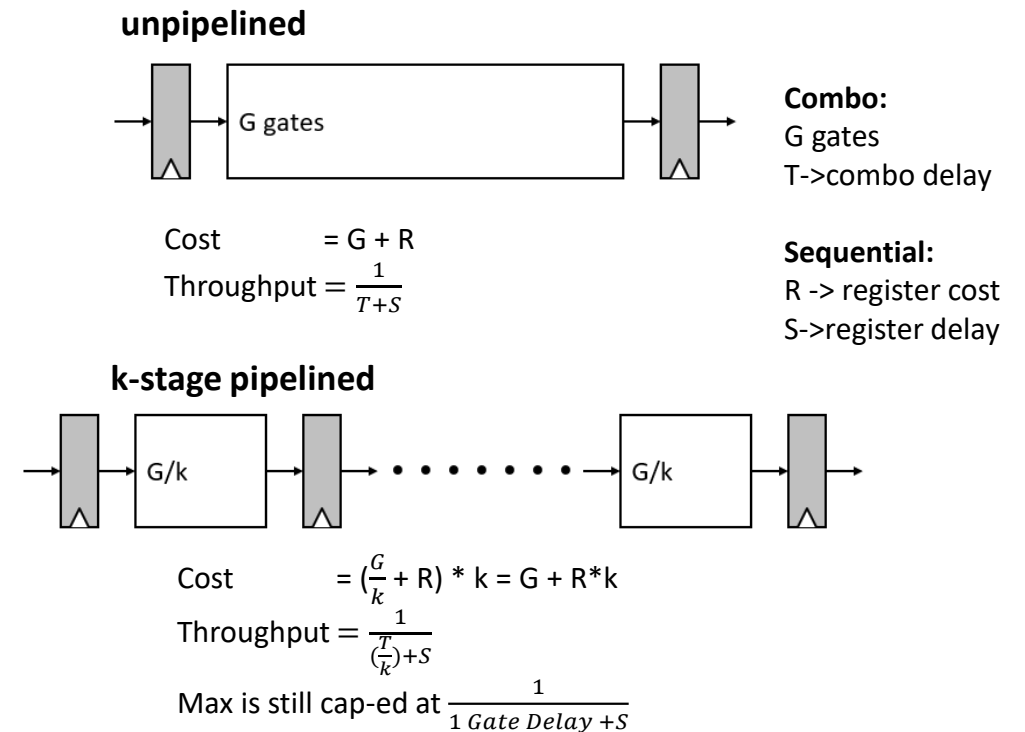
Area:

- Much better hardware utilization

Pipelining : Cost that we pay?

Can we keep on increasing pipe stages to achieve higher clock frequencies?

- Pipeline registers' hardware cost
 - Delay + area
- Instruction latency has increased
 - Especially if stages are not perfectly balanced
- Hardware complexity
 - To resolve pipelining dependencies (hazards)
- Area has increased
 - But is better utilized
- Power has increased



Showing the case where stages are not balanced

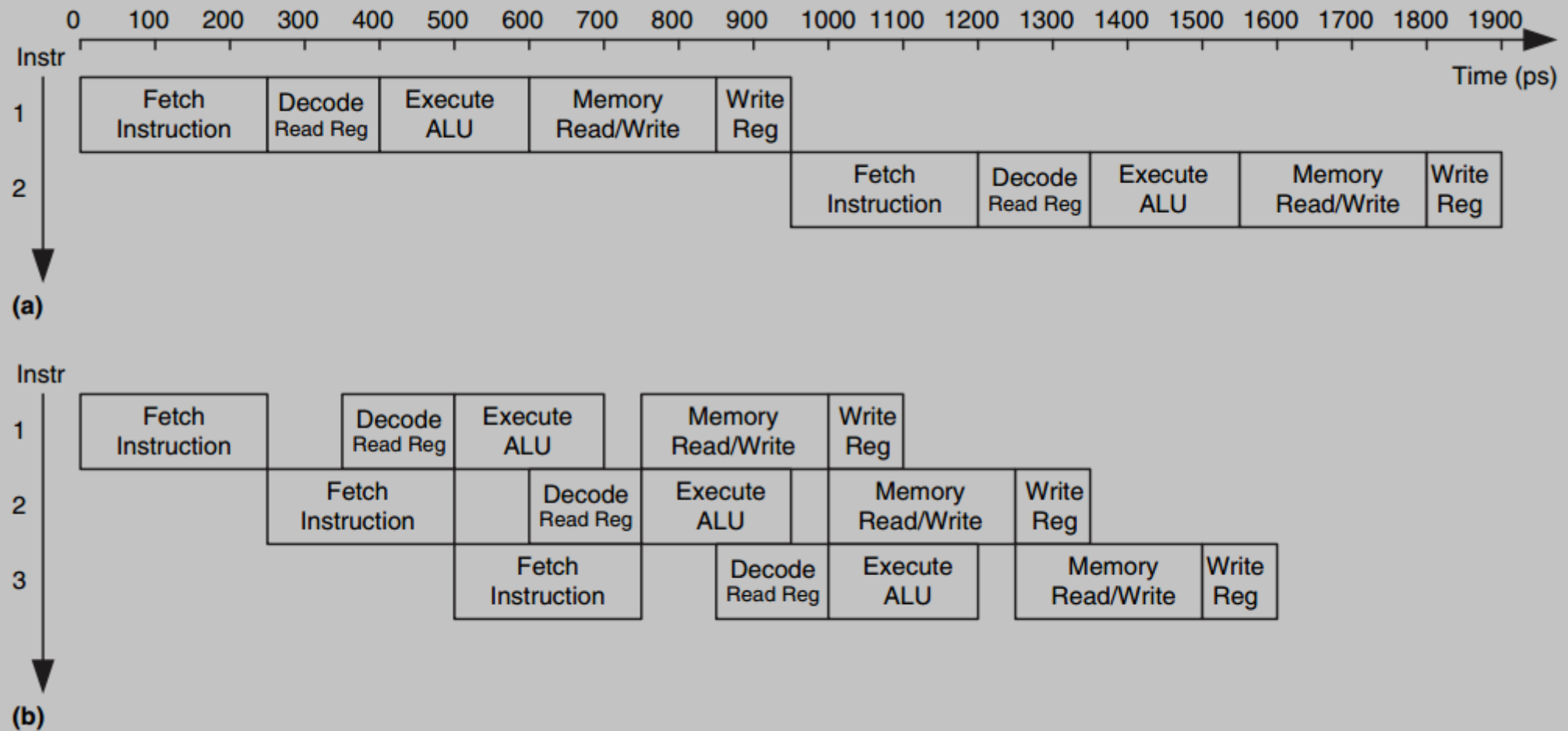
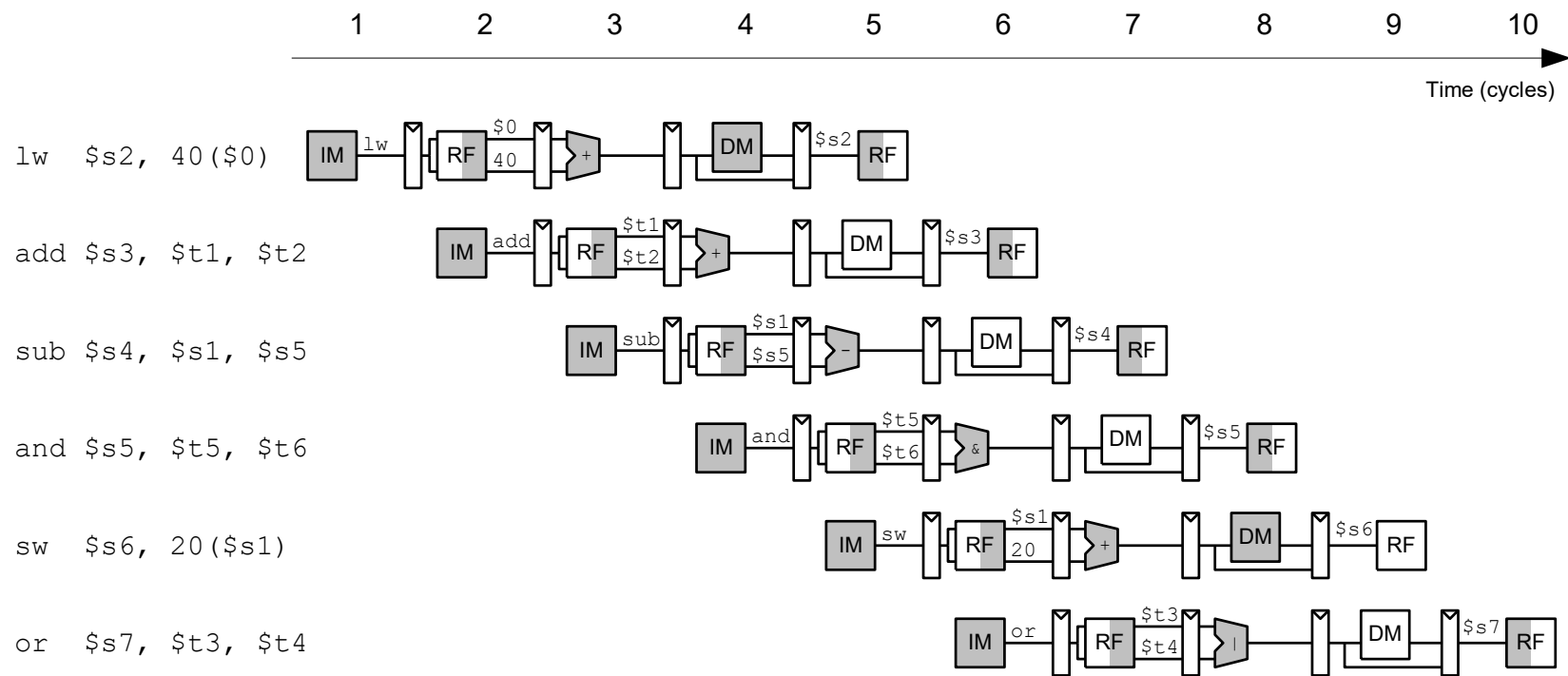
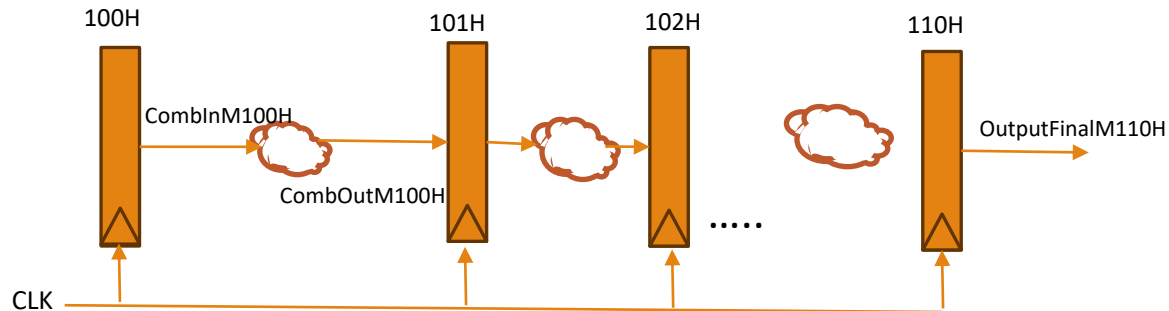


Figure 7.43 Timing diagrams: (a) single-cycle processor, (b) pipelined processor



Pipelining: Verilog Code

```
define FF(output reg[DATA_WIDTH-1:0] out, input reg [DATA_WIDTH-1:0] in, input clock,input rst)
always @(posedge clock)
begin
    if (rst) out <= '0;
    else     out <= i;
end
end
```



Diagram/code uses some notations

```
reg [DATA_WIDTH-1:0] CombInM100H, CombInM101H, .... OutputFinalM110H;
wire [DATA_WIDTH-1:0] CombOutM100H, CombOutM101H, ... CombOutM109H;

//Pipelining Stage 100H -----
//COMBO LOGIC
always @(*) begin
    ....
end

assign CombOutM100H = combo_logic ;

//PIPELINING REGISTER
`FF(CombInM101H, CombOutM100H, clk, rst)
//-----

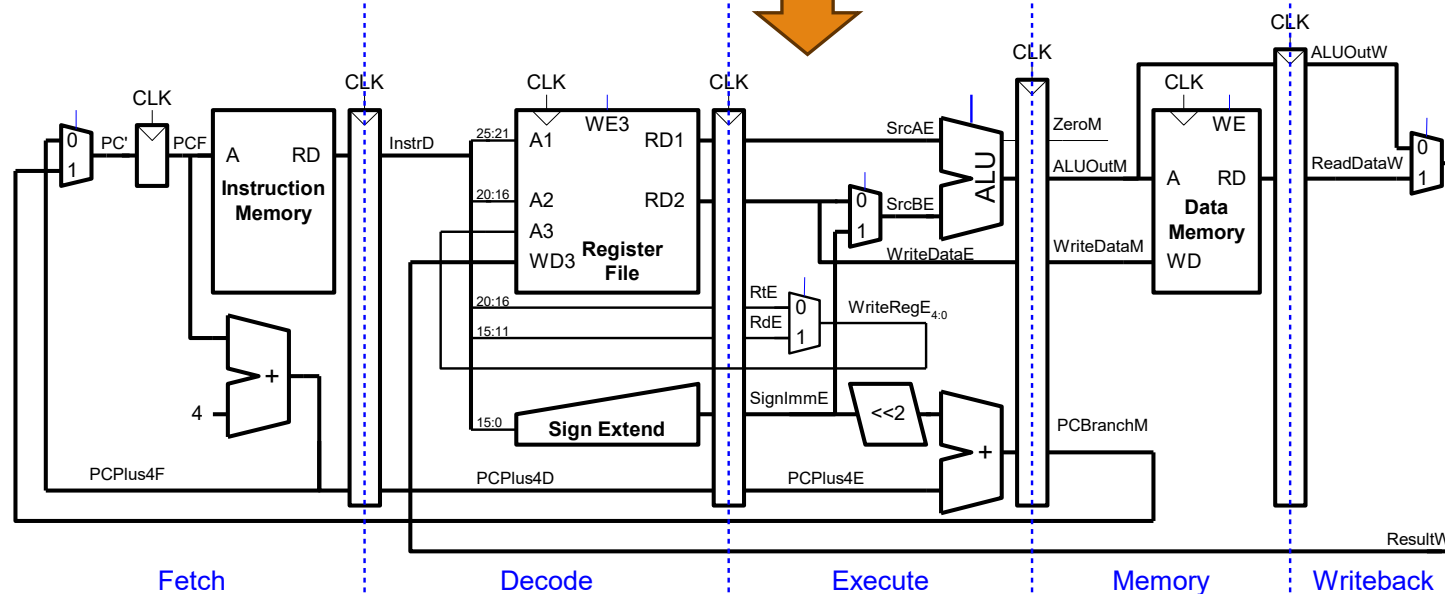
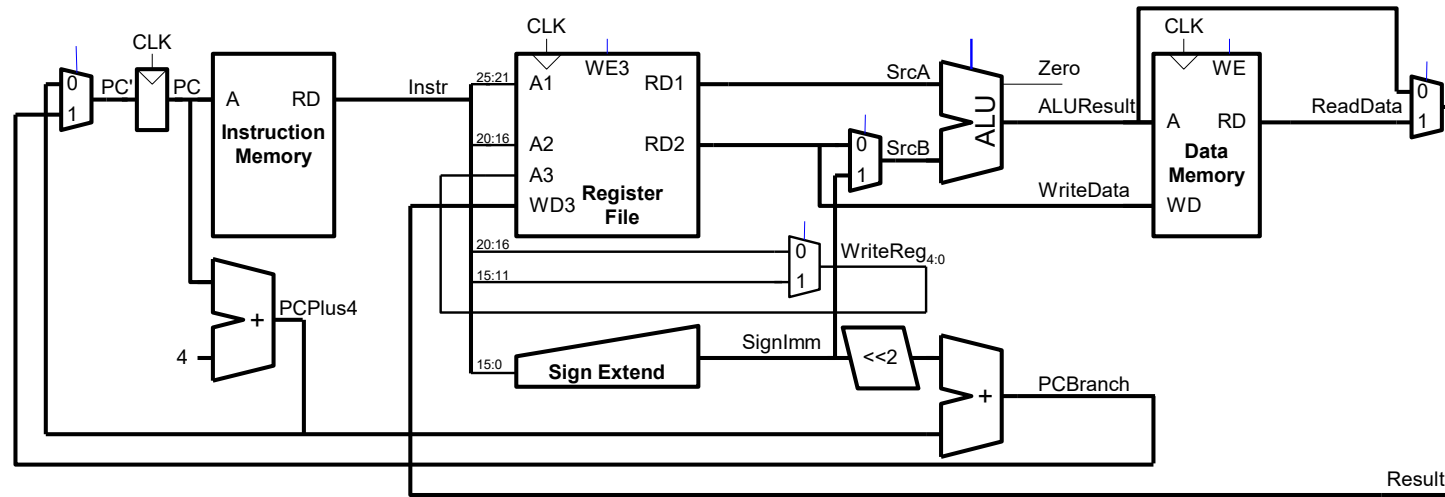
//Pipelining Stage 101H
//COMBO LOGIC
always @(*) begin
    ....
end

assign CombOutM101H = combo_logic ;

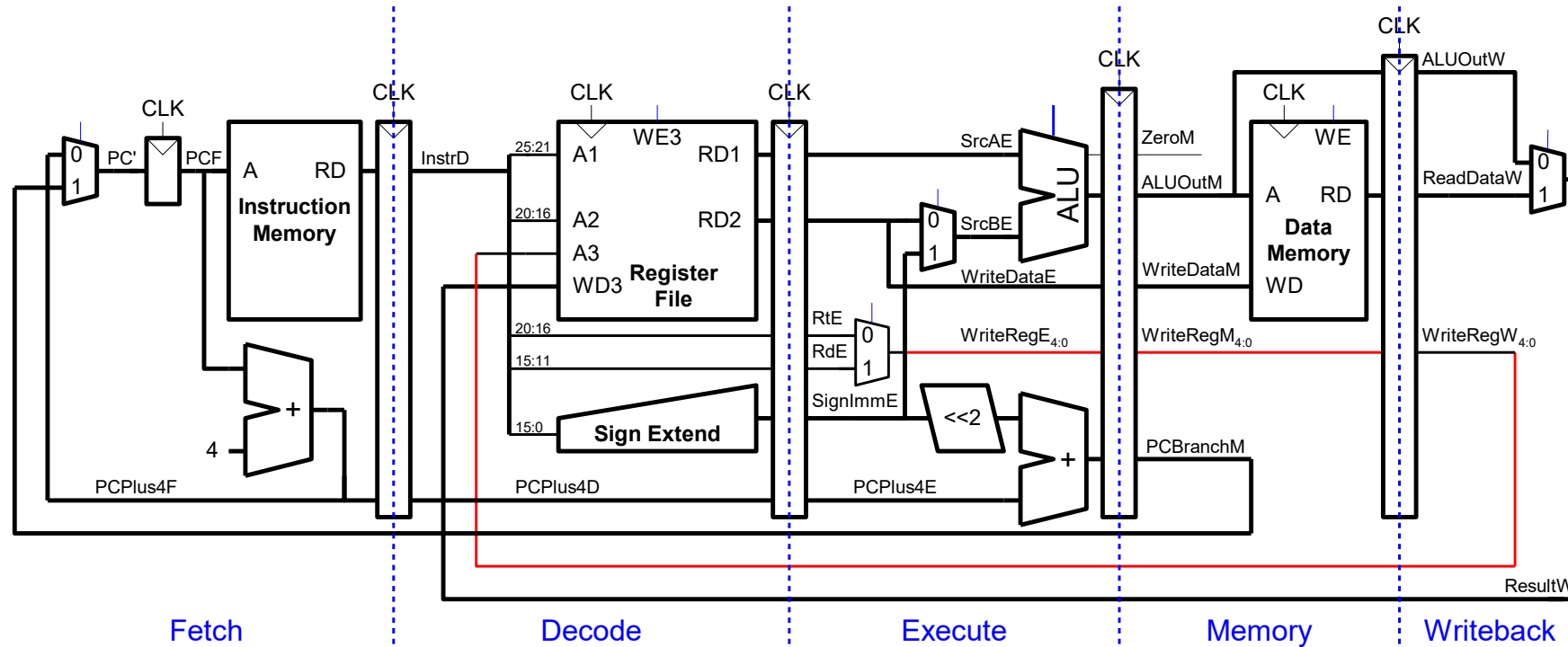
//PIPELINING REGISTER
`FF(CombInM102H, CombOutM101H, clk, rst)
//-----

..
`FF(OutputFinalM110H, CombOutM109H, clk, rst)
```

Pipelined Datapath

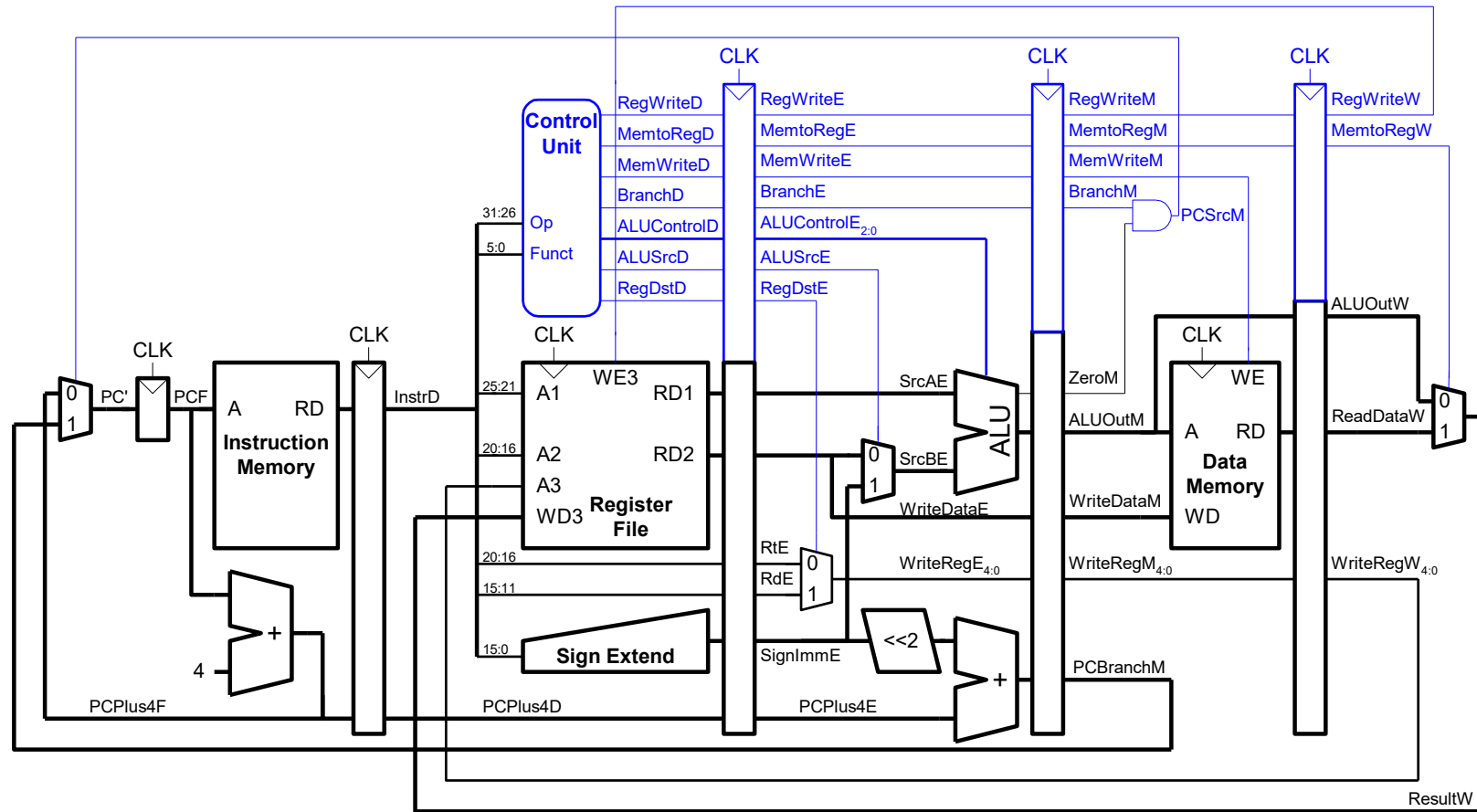


Corrected Pipelined Datapath



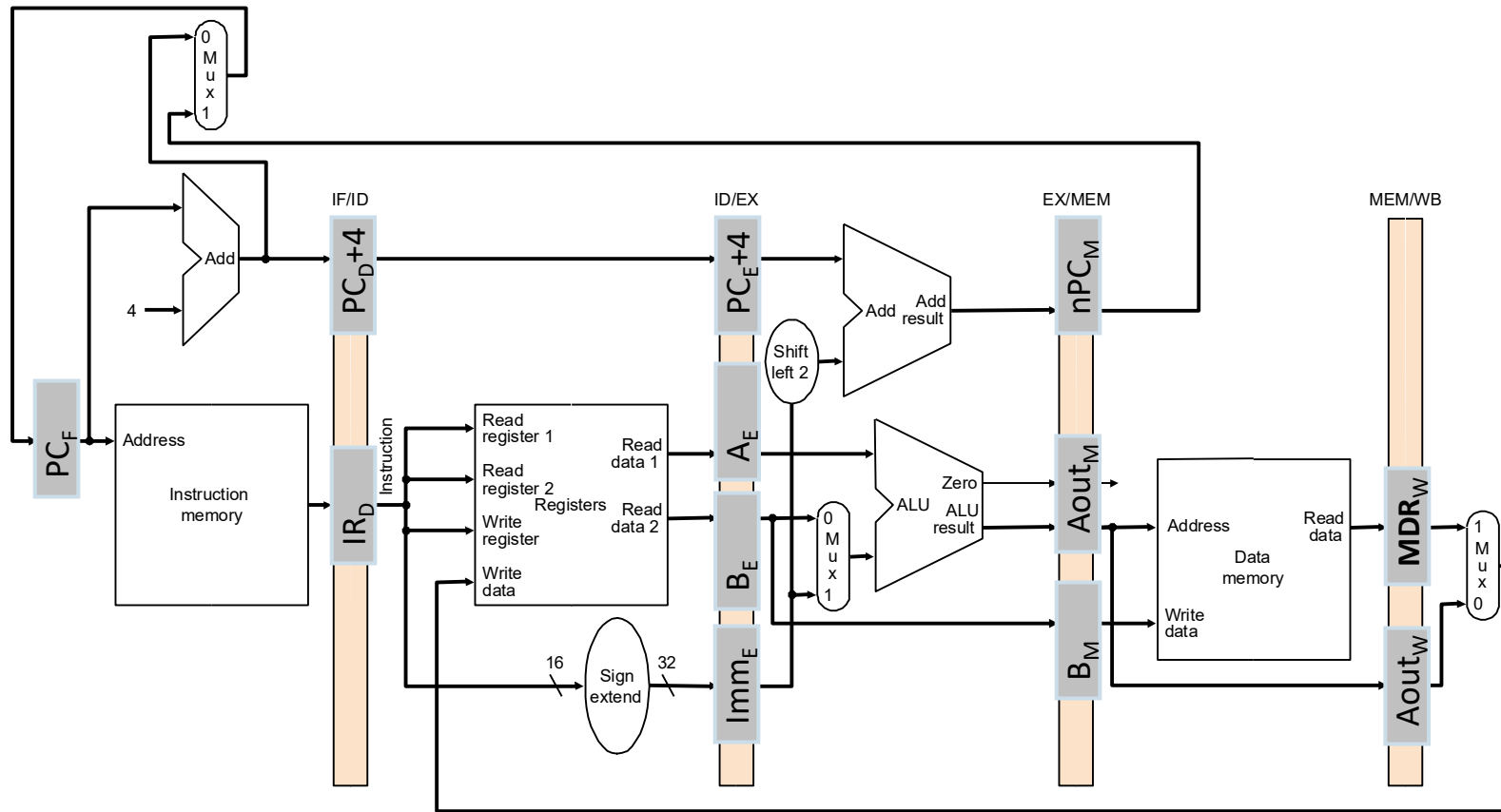
WriteReg must arrive at same time as Result

Pipelined Control

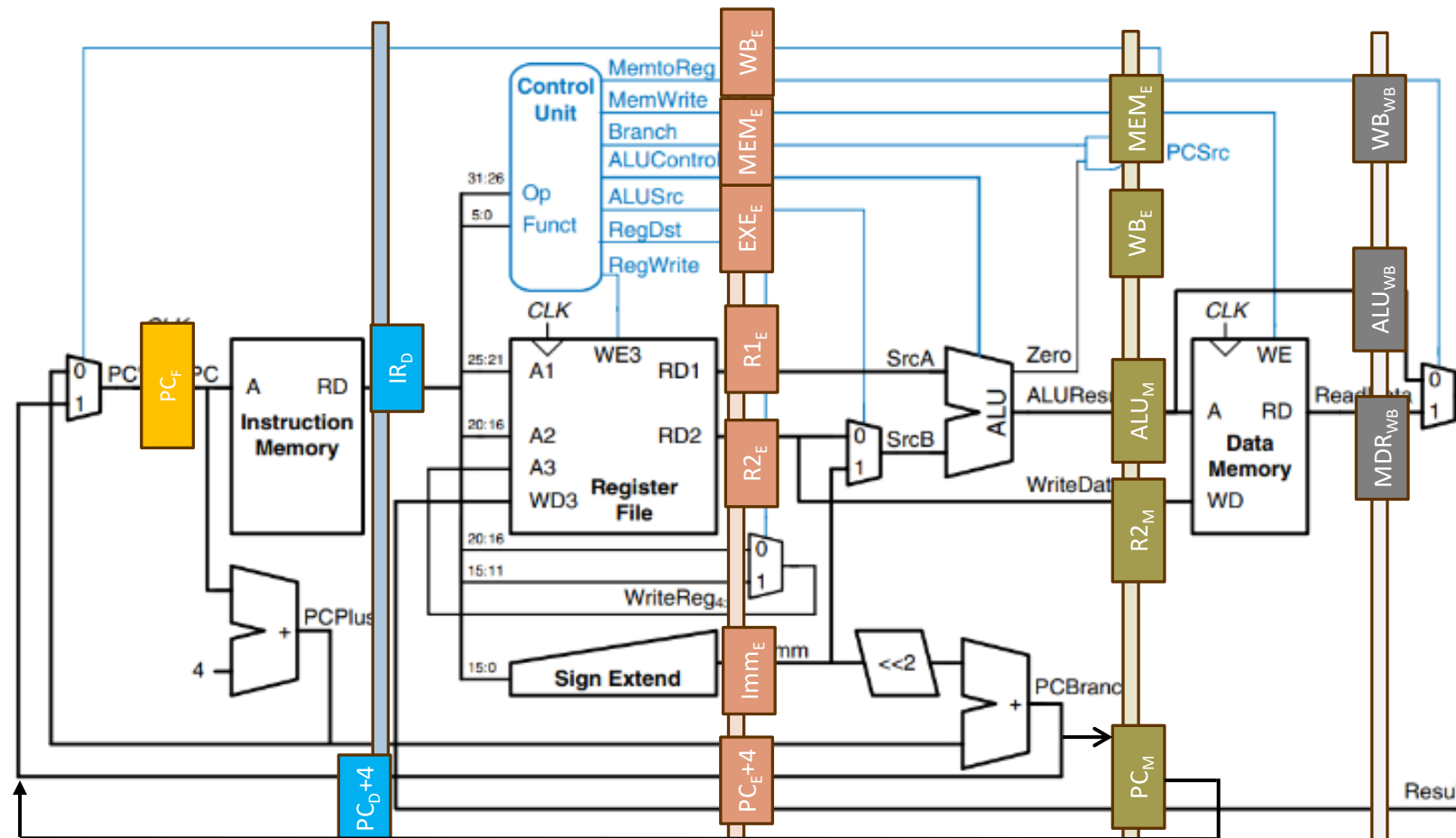


- Same control unit as single-cycle processor
- Control delayed to proper pipeline stage

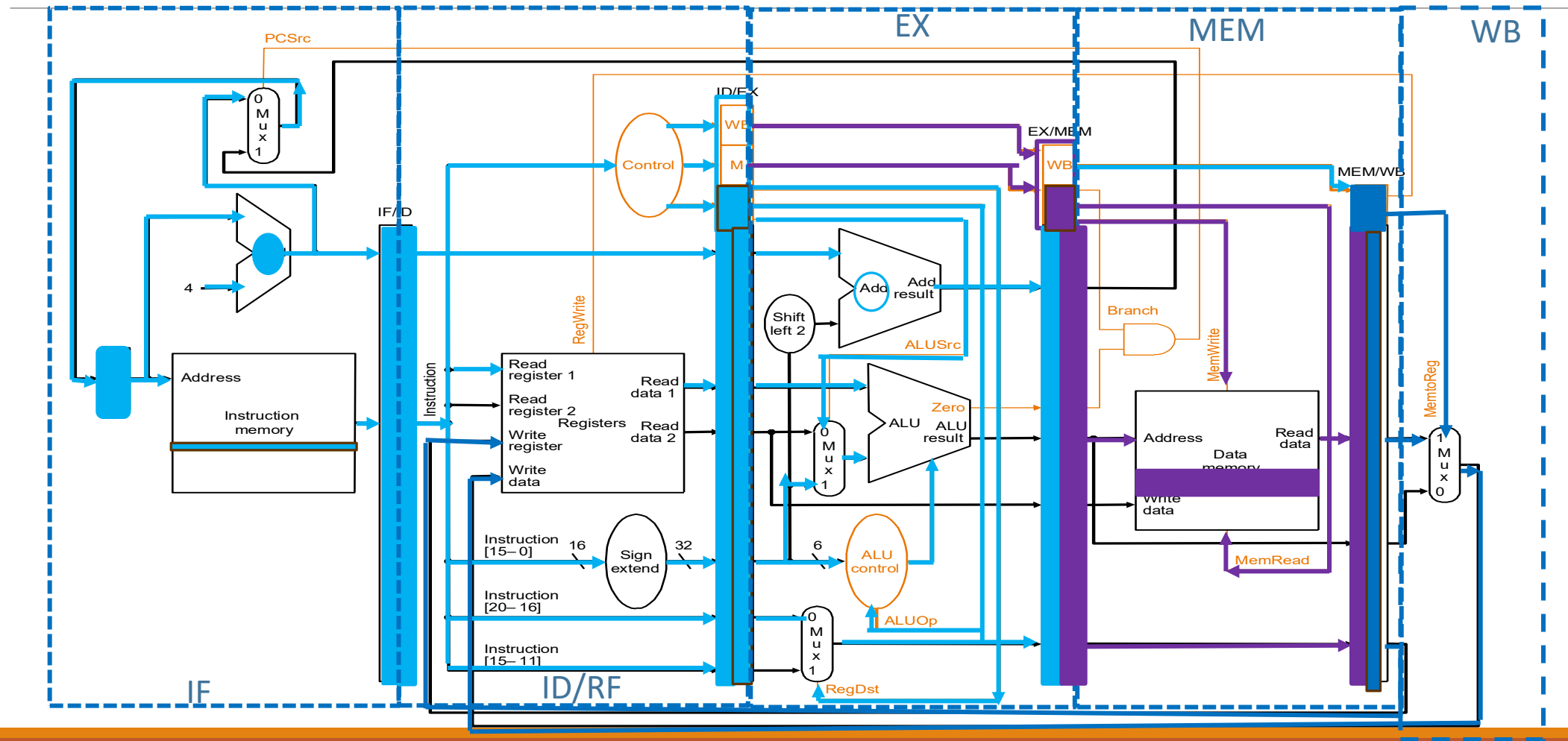
MIPS : Pipelined Datapath



MIPS : Pipelined Design Notation

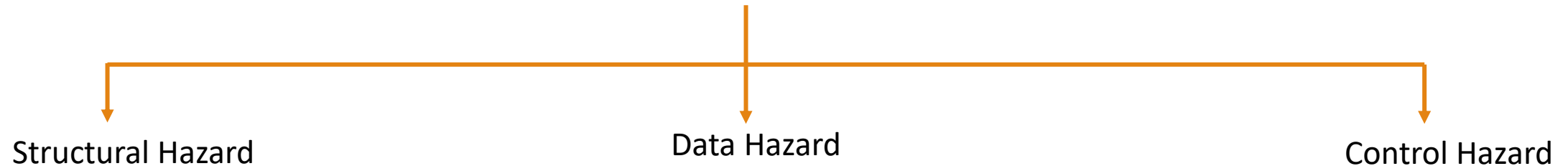


Pipelined Design: lw example



Pipeline Hazards/Dependencies

When an instruction's execution cannot make progress due to a dependence on earlier instruction's execution

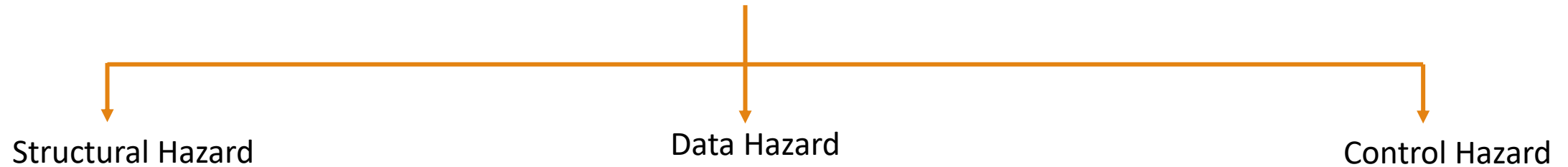


- When does it happen?
 - Resource conflict (resource not fully pipelined, limited ports to mem structures, etc)
 - Example : Let's say RF has a single port to read or write. Decode vs WB stage
- How do you resolve it?
 - Deal with trade-offs
 - Duplicate hardware, increase the number of ports accessing mem structures
 - Stall the conflicting instruction till resource is available – be careful in selecting which instruction to stall

There is always a work-around but it always has a trade-off that you need to assess!!

Pipeline Hazards/Dependencies

When an instruction's execution cannot make progress due to a dependence on earlier instruction's execution



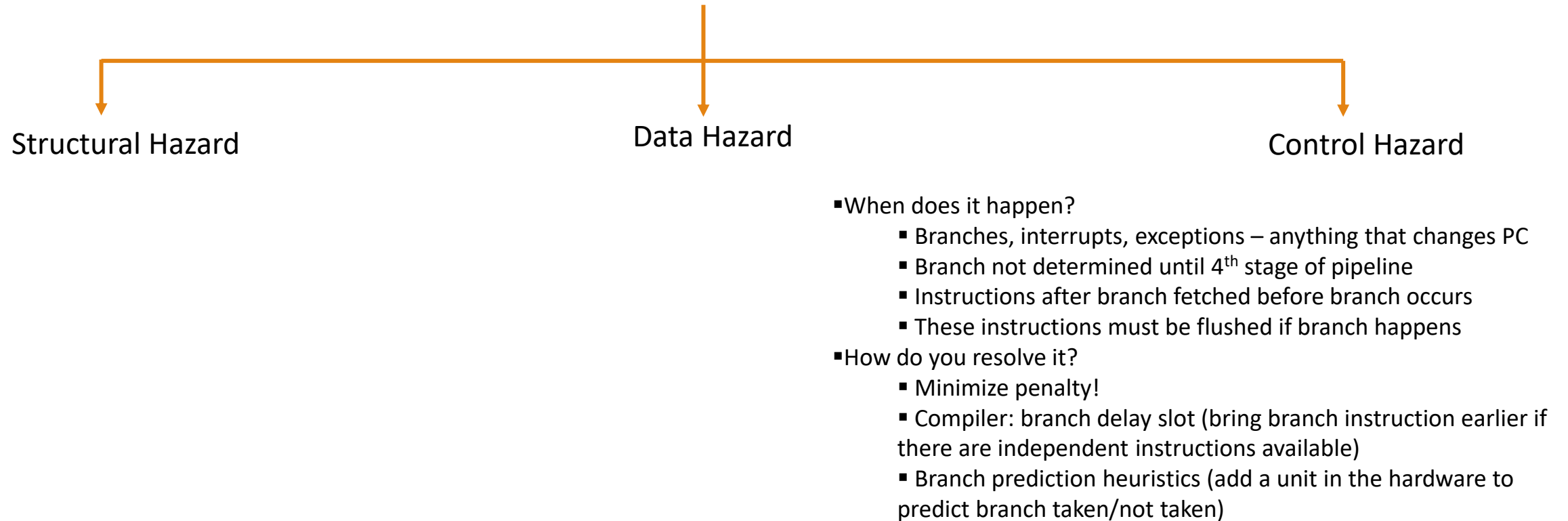
$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_5 \leftarrow r_3 \text{ op } r_4$

Instr's exec is dependent on earlier instr's result.
Register value not yet written back to register file

There is always a work-around but it always has a trade-off that you need to assess!!

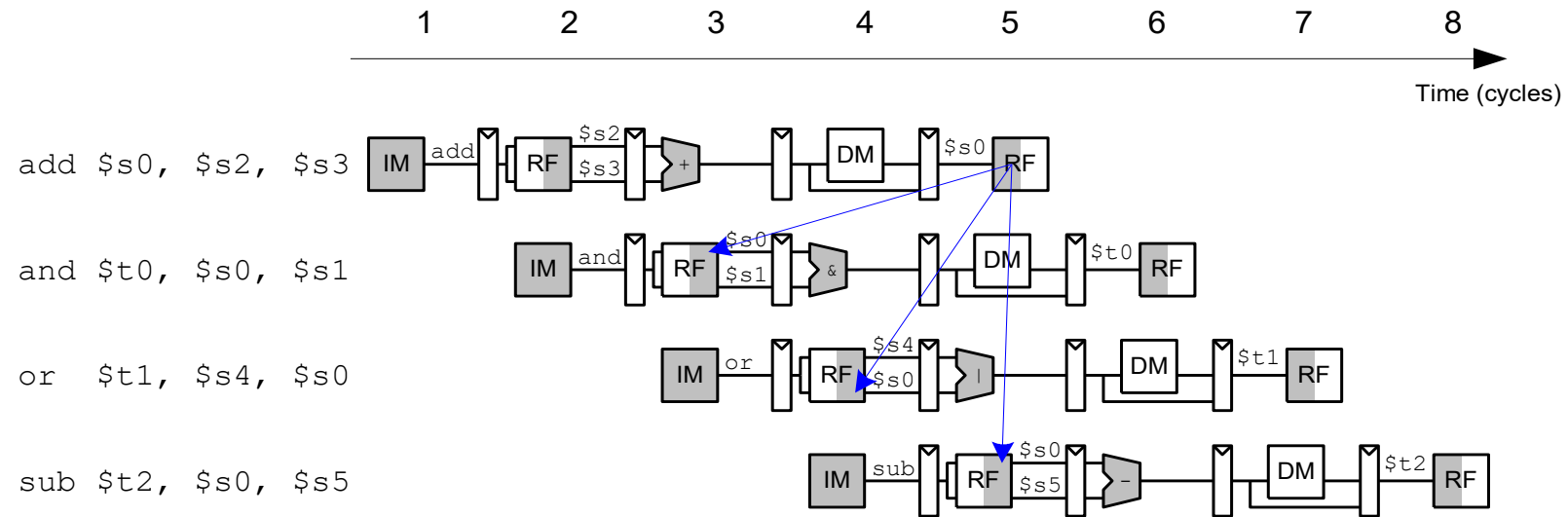
Pipeline Hazards/Dependencies

When an instruction's execution cannot make progress due to a dependence on earlier instruction's execution



There is always a work-around but it always has a trade-off that you need to assess!!

Data Hazard



Handling Data Hazards

Insert `nops` in code at compile time

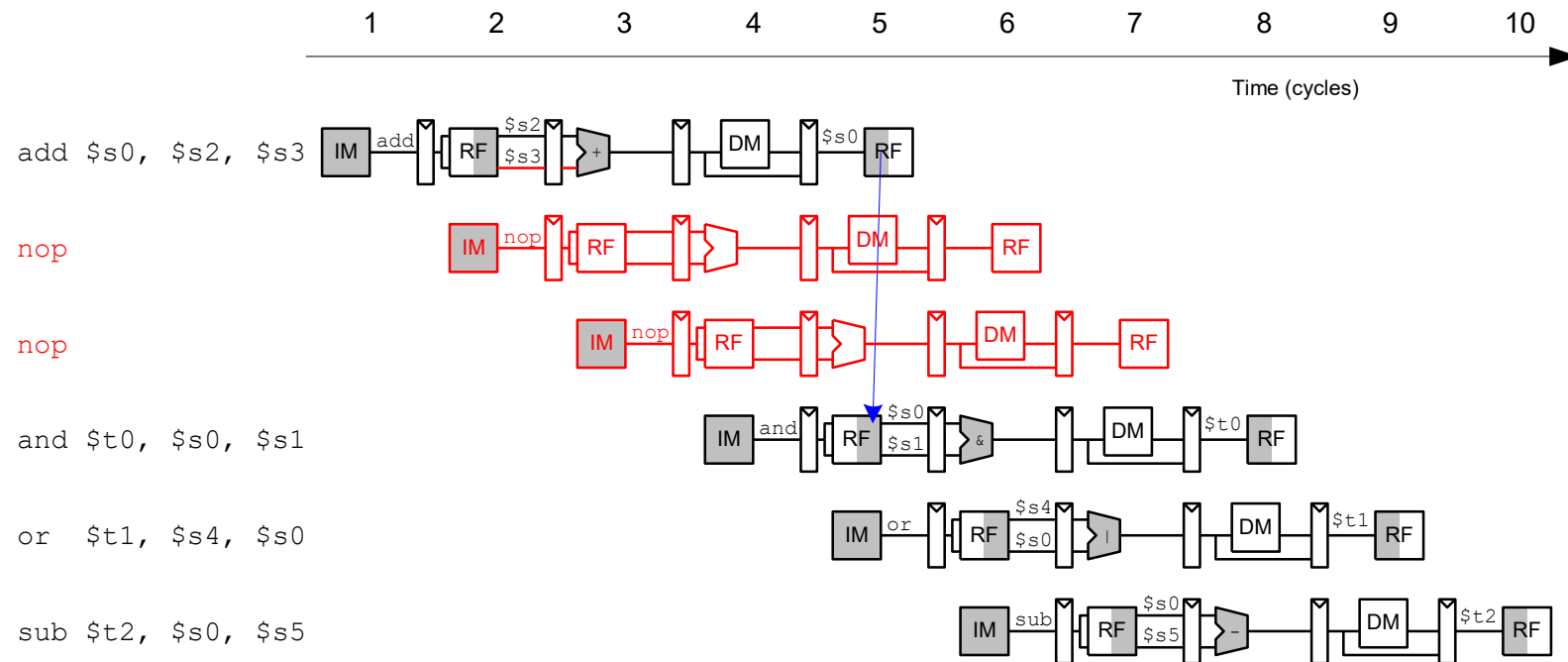
Rearrange code at compile time

Forward data at run time

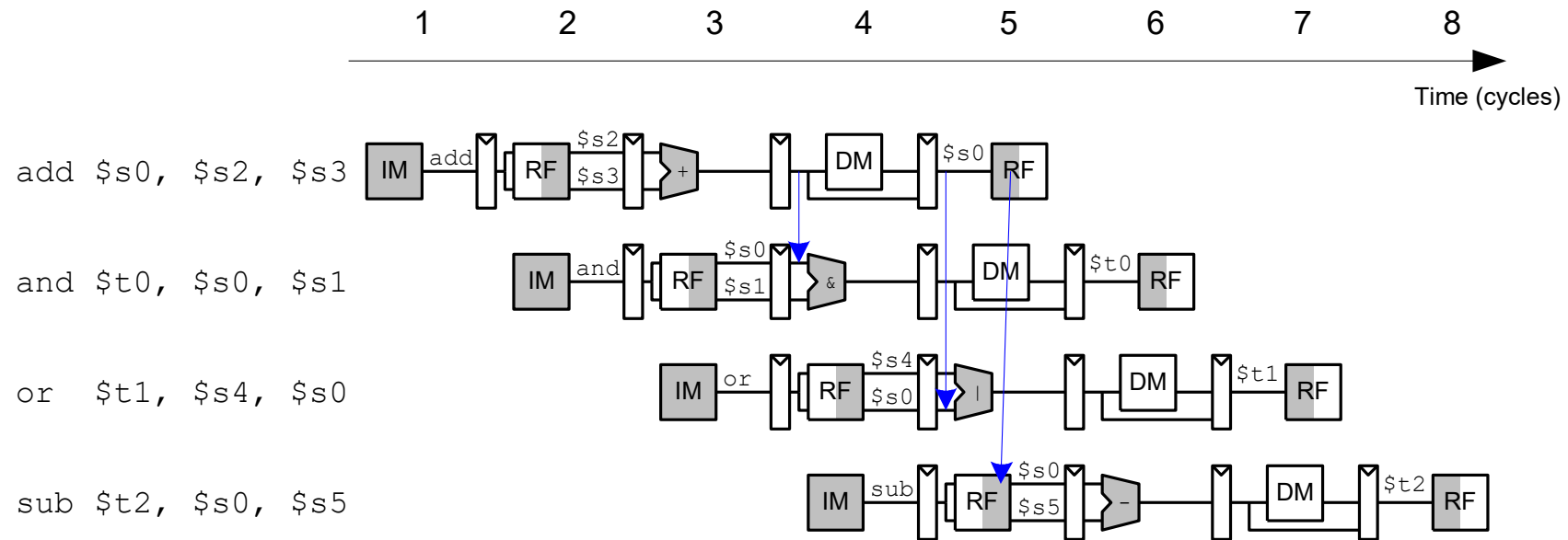
Stall the processor at run time

Compile-Time Hazard Elimination

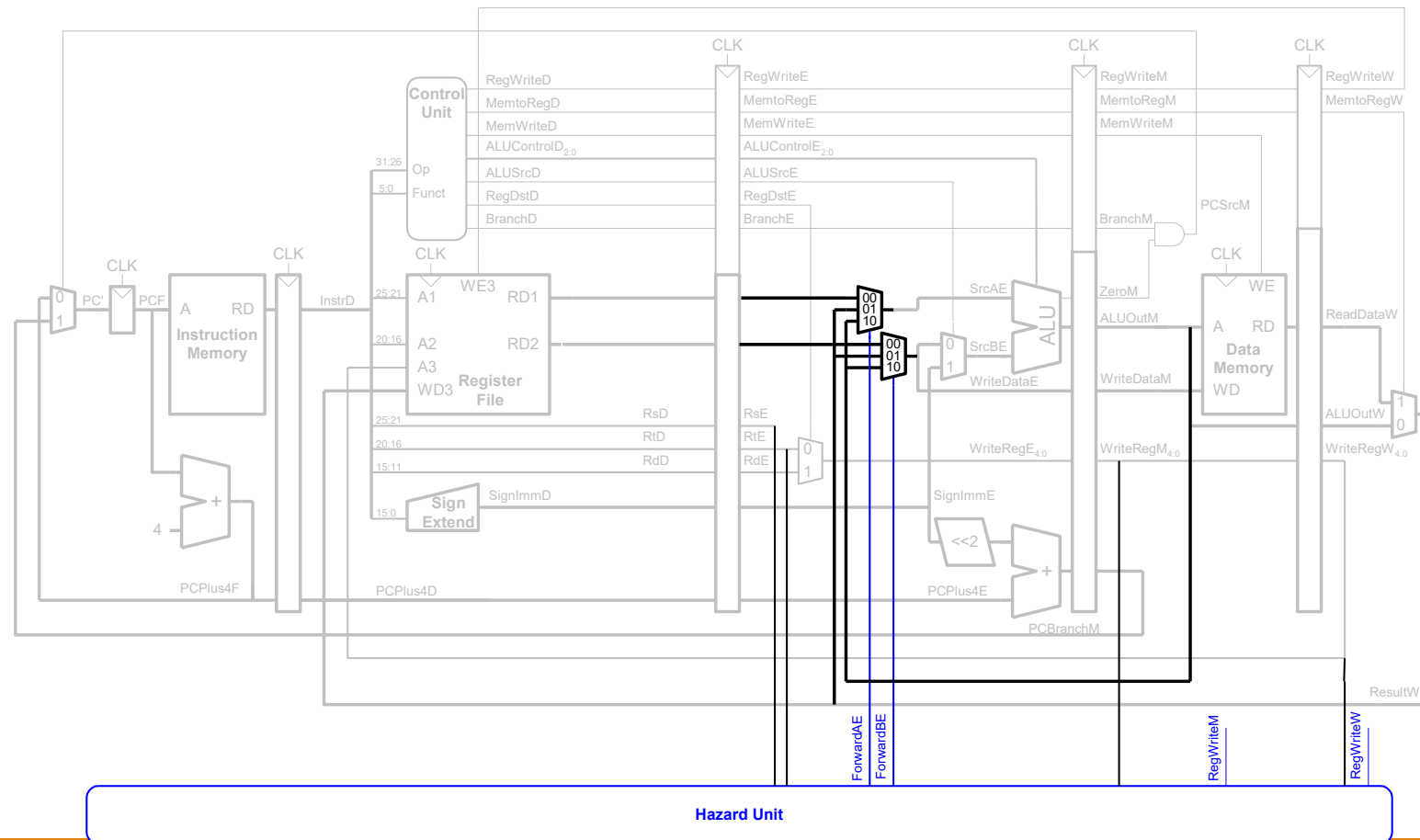
- Insert enough nops for result to be ready
- Or move independent useful instructions forward



Data Forwarding



Data Forwarding Implementation



Data Forwarding Implementation

Forward to Execute stage from either:

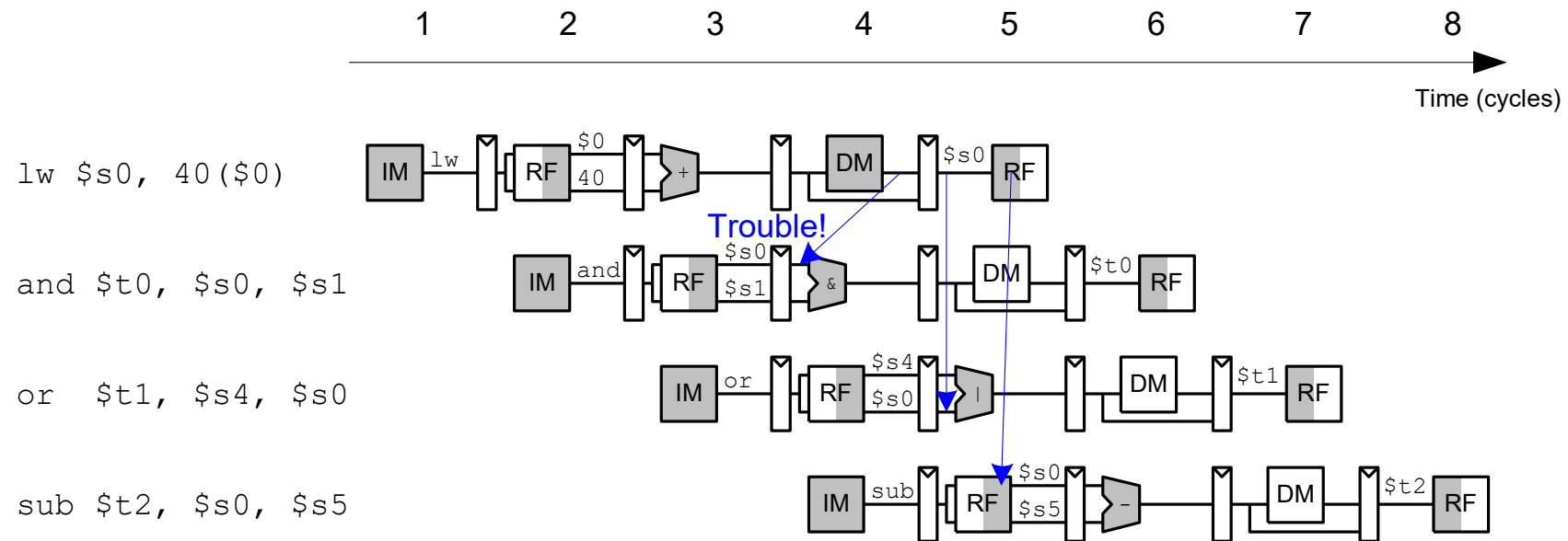
- Memory stage or
- Writeback stage

Forwarding logic for *ForwardAE*:

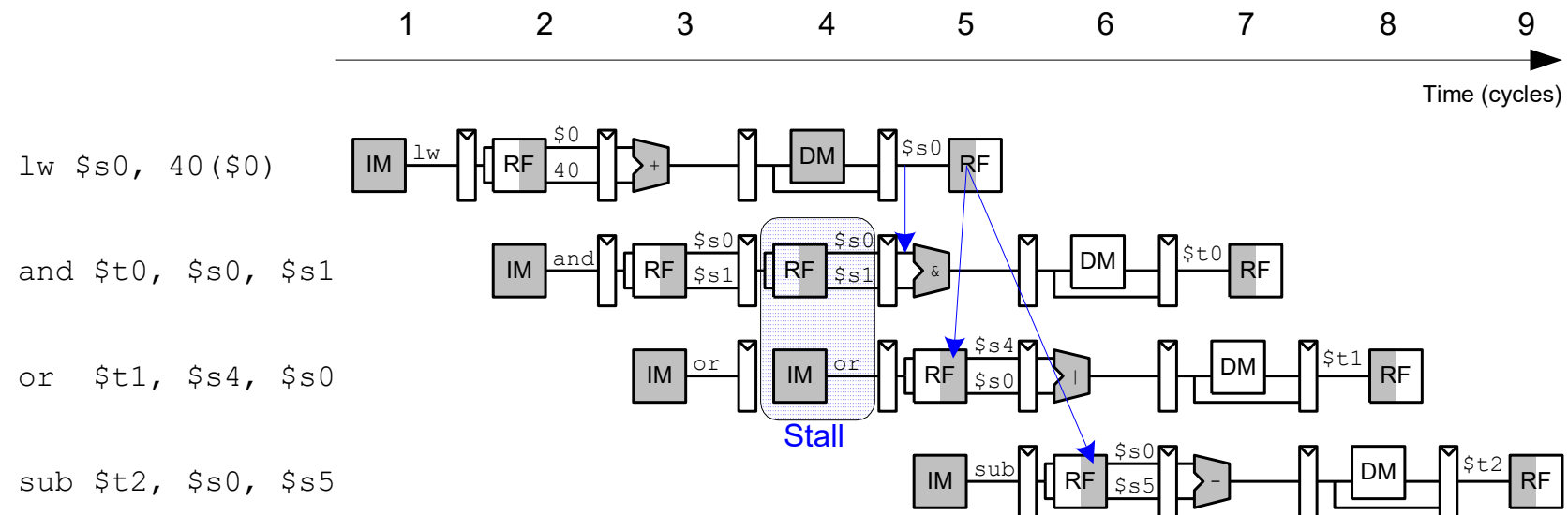
```
if      ((rsE != 0) AND (rsE == WriteRegM) AND RegWriteM)
    then ForwardAE = 10
else if ((rsE != 0) AND (rsE == WriteRegW) AND RegWriteW)
    then ForwardAE = 01
else    ForwardAE = 00
```

Forwarding logic for *ForwardBE* same, but replace *rsE* with *rtE*

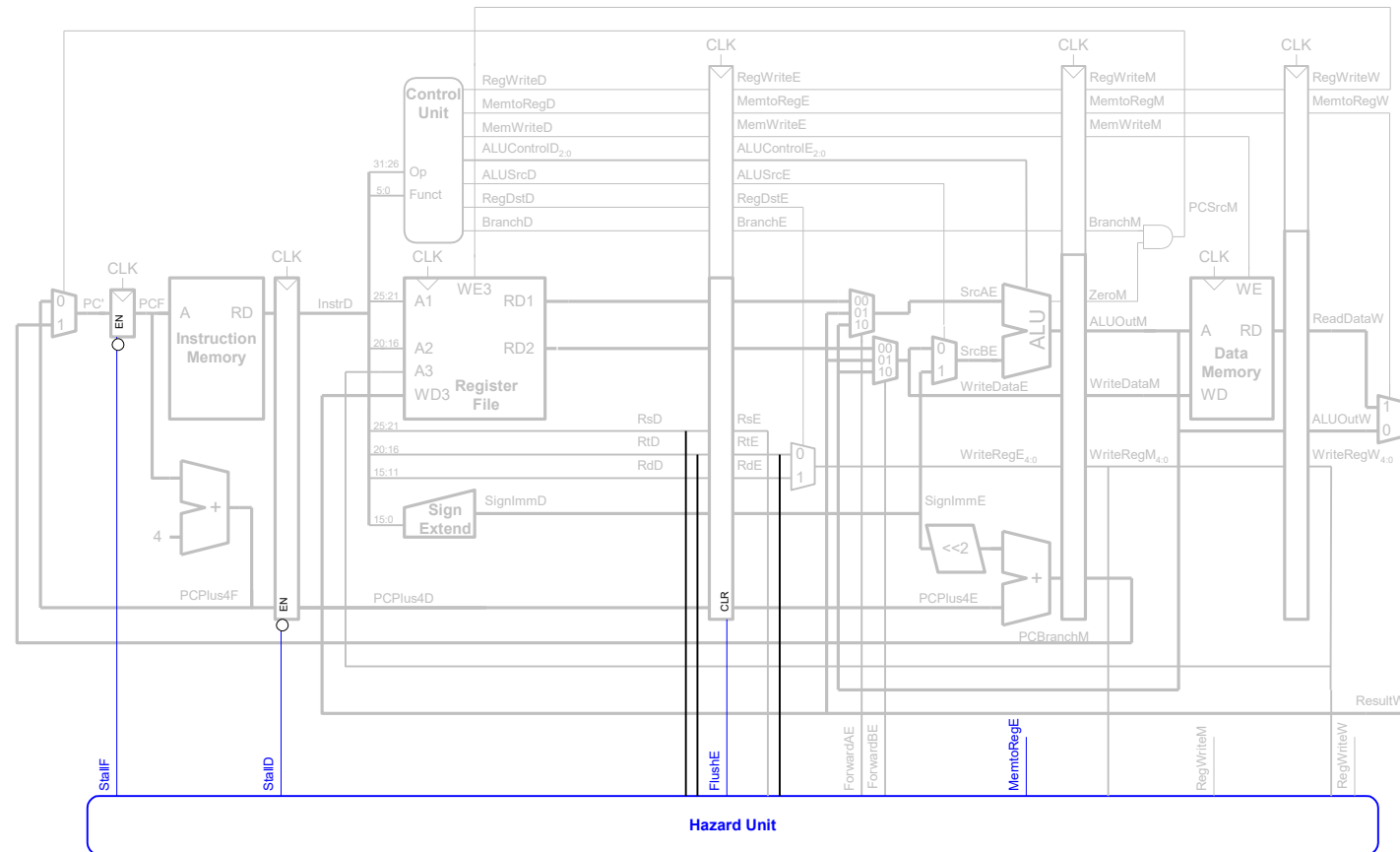
Stalling



Stalling



Stalling Implementation

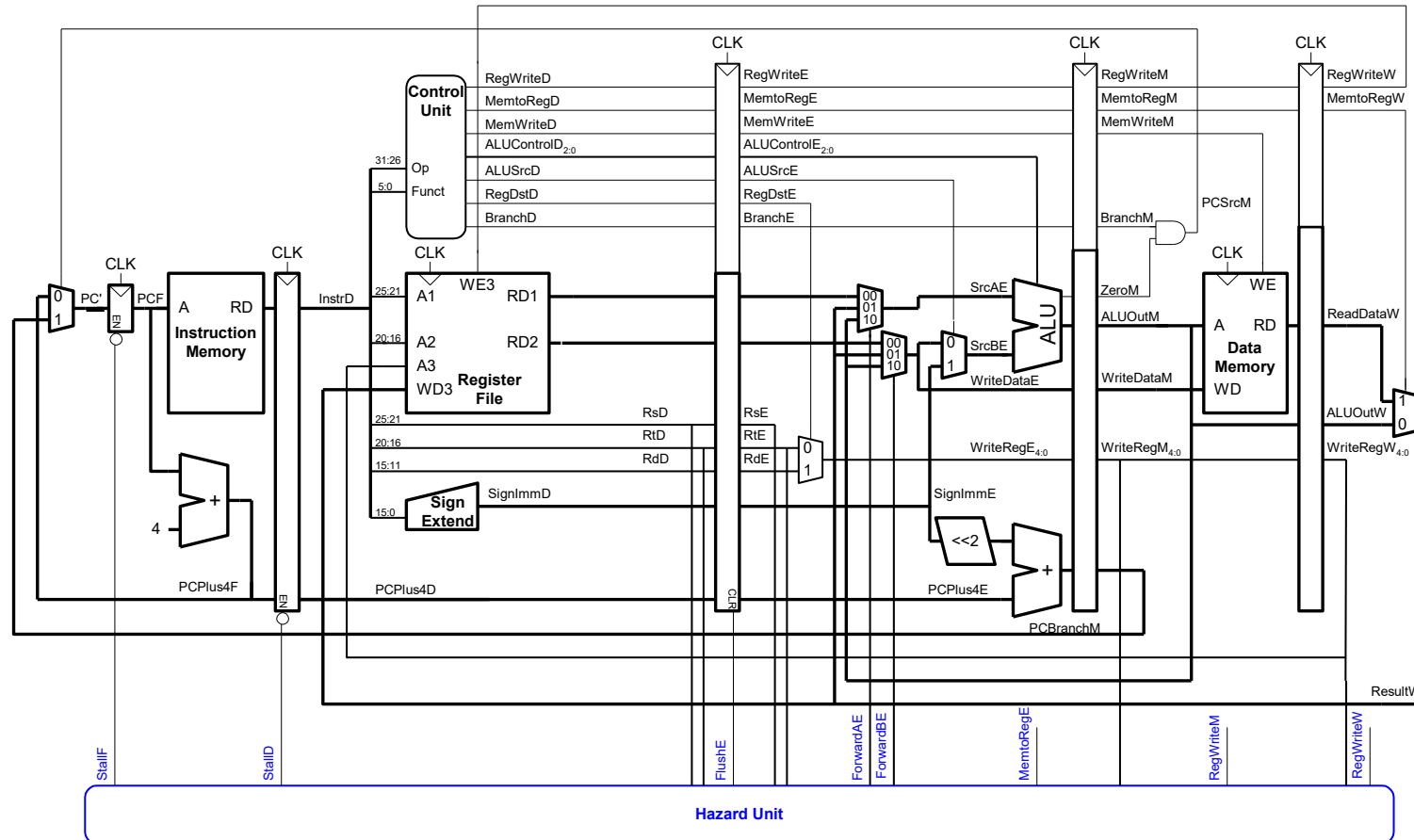


Stalling Implementation

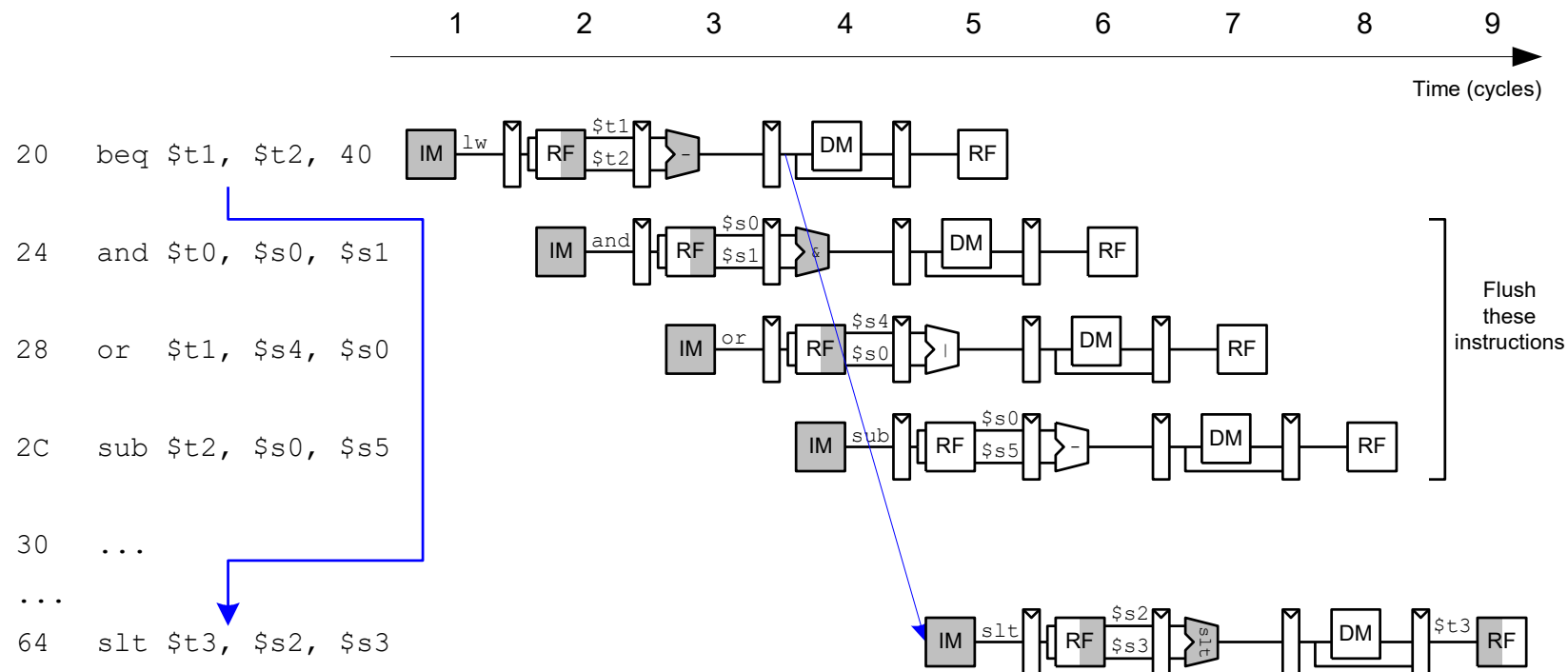
lwstall =
 ((*rsD*==*rtE*) OR (*rtD*==*rtE*)) AND *MemtoRegE*

StallF = *StallD* = *FlushE* = *lwstall*

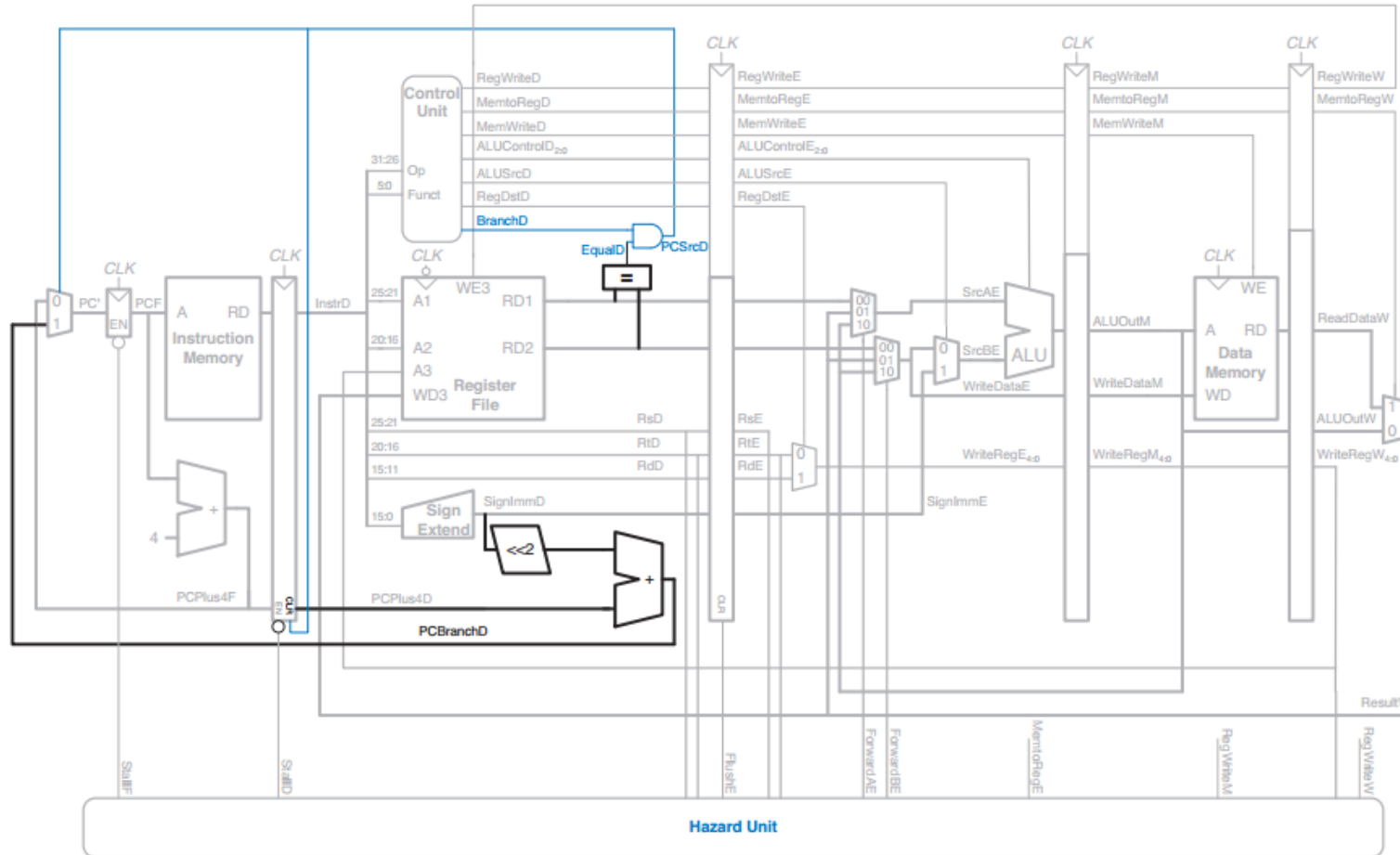
Control Hazards: Original Pipeline



Control Hazards



Early Branch Resolution

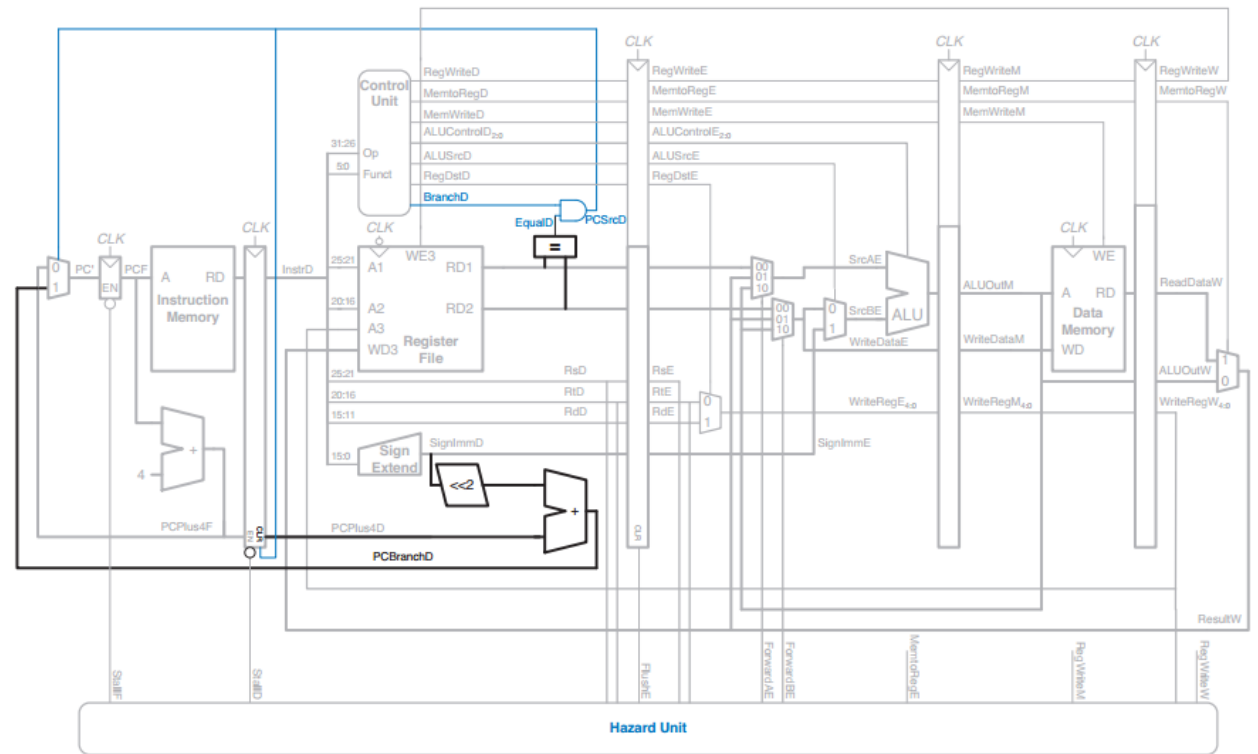


Early Branch Resolution

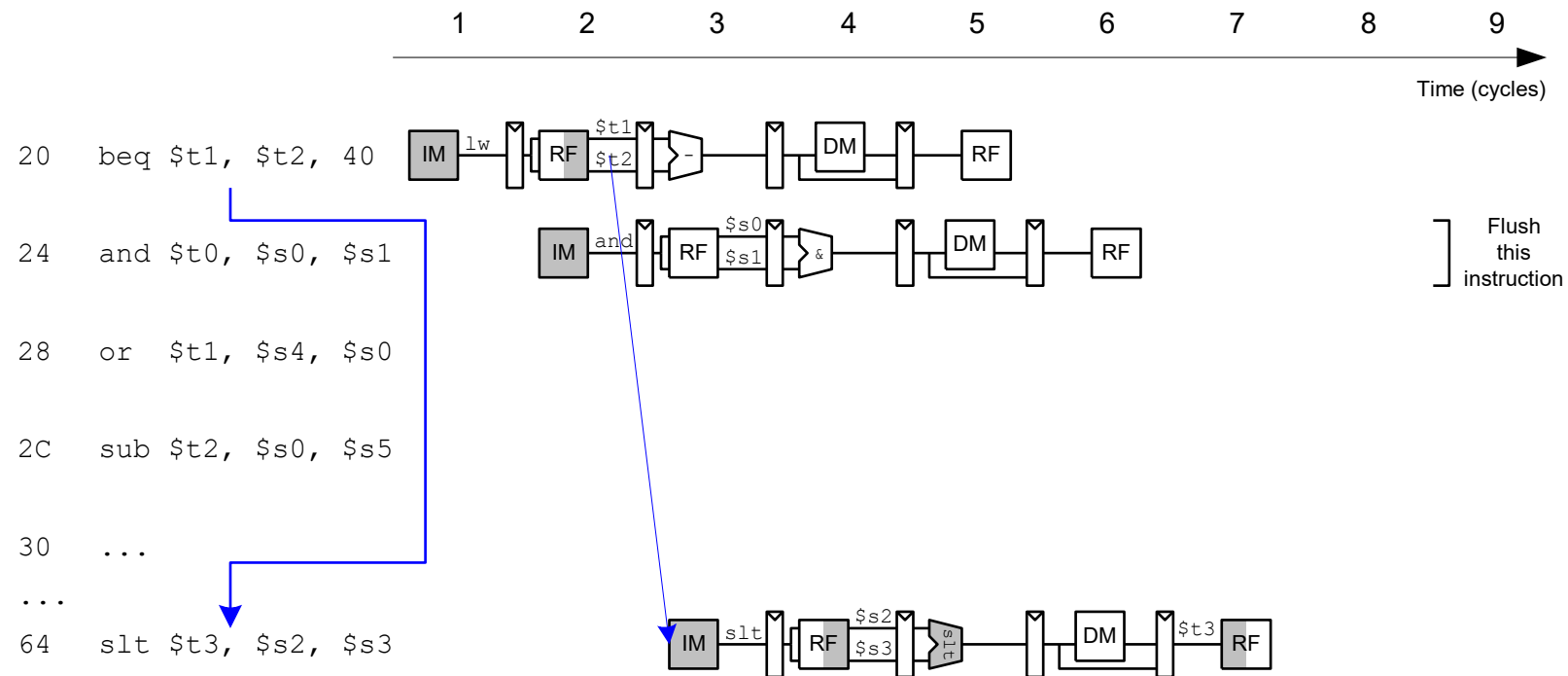
Introduced another data hazard in Decode stage.

If one of the source operands for the branch was computed by a previous instruction and has not yet been written into the RF, the branch will read the wrong operand value from the RF.

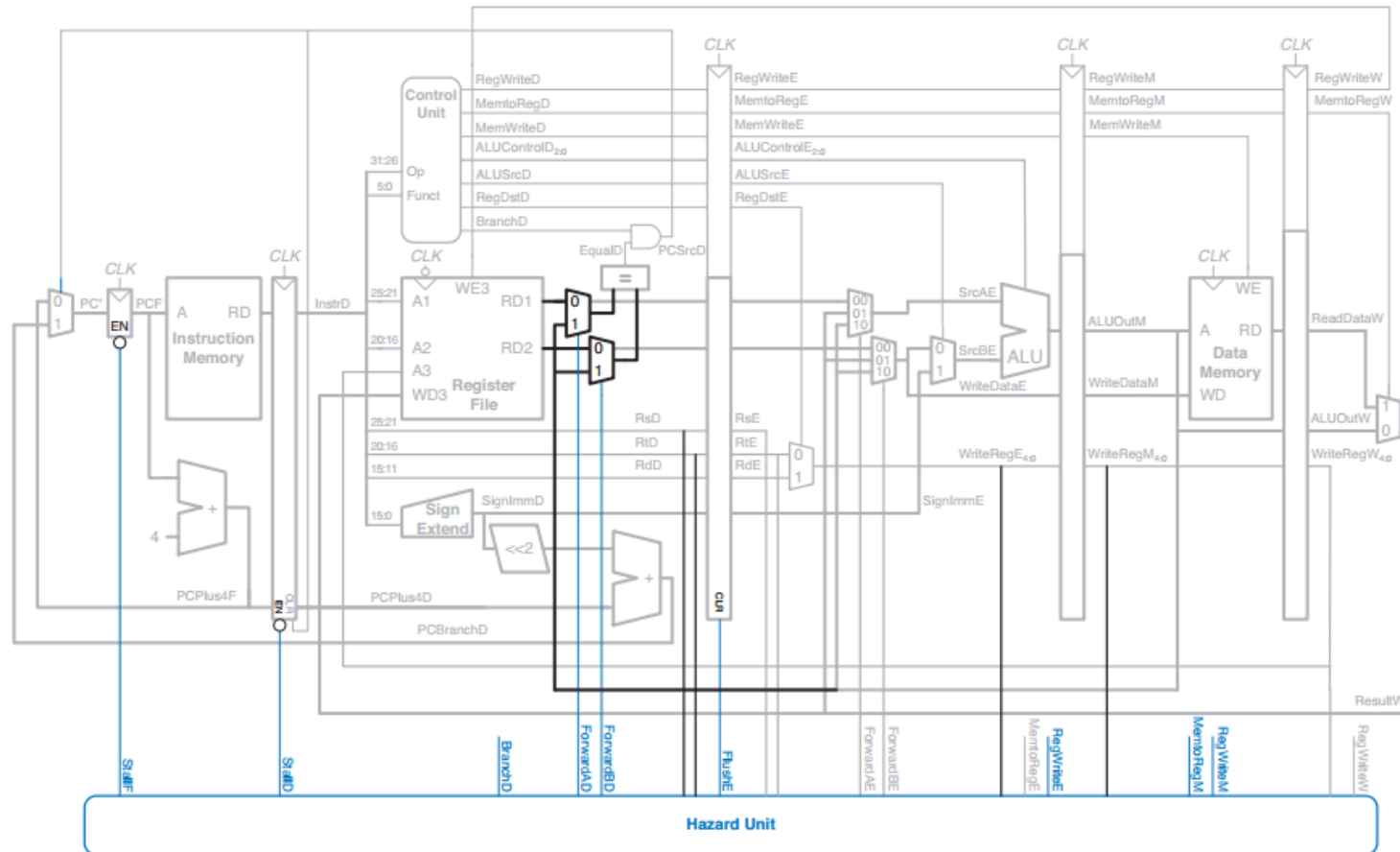
Solve this by forwarding the correct value if it is available or by stalling the pipeline until the data is ready



Early Branch Resolution



Earlier Branch Resolution: Fixing Data Hazard



Control Forwarding & Stalling Logic

Forwarding logic:

ForwardAD = (*rsD* != 0) AND (*rsD* == *WriteRegM*) AND *RegWriteM*

ForwardBD = (*rtD* != 0) AND (*rtD* == *WriteRegM*) AND *RegWriteM*

Stalling logic:

branchstall = *BranchD* AND

[*RegWriteE* AND ((*WriteRegE* == *rsD*) OR (*WriteRegE* == *rtD*))

OR

[*MemtoRegM* AND ((*WriteRegM* == *rsD*) OR (*WriteRegM* == *rtD*))]

StallF = **StallD** = *FlushE* = *lwstall* OR *branchstall*

Branch Prediction

Guess whether branch will be taken

- Backward branches are usually taken (loops)
- Consider history to improve guess

Good prediction reduces fraction of branches requiring a flush

Static branch prediction:

- Check direction of branch (forward or backward)
- If backward, predict taken
- Else, predict not taken

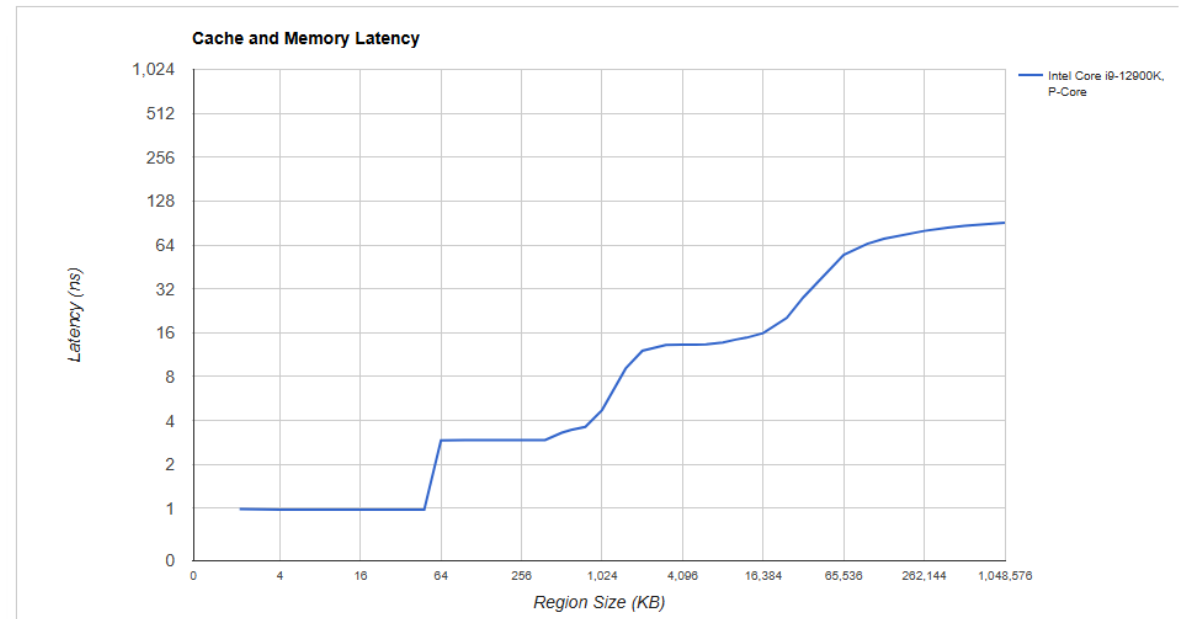
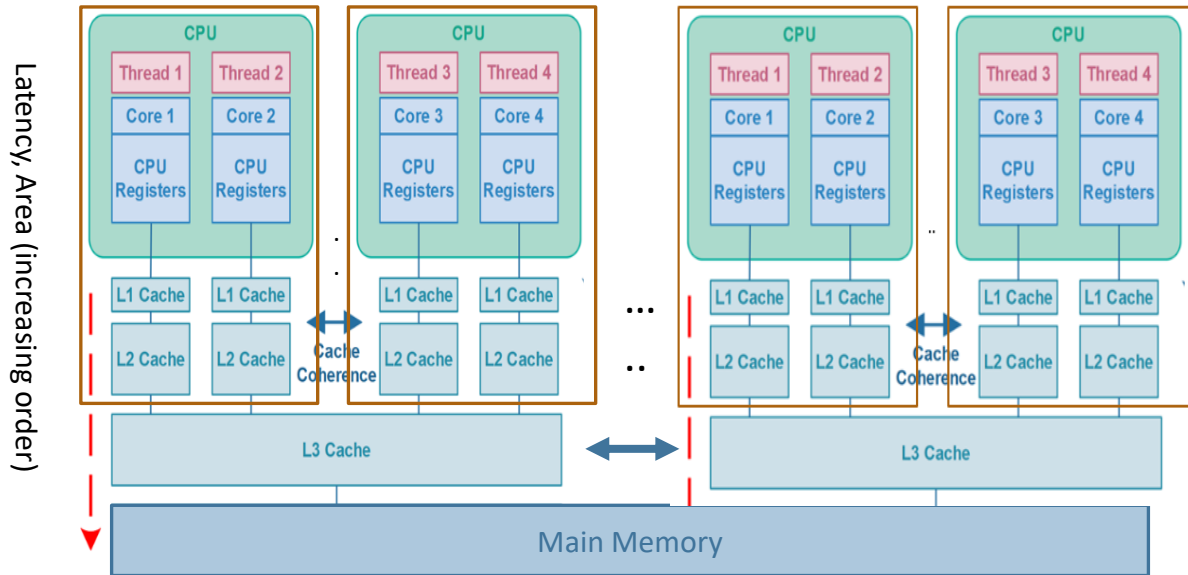
Dynamic branch prediction:

- Keep history of last (several hundred) branches in *branch target buffer*, record:
 - Branch destination
 - Whether branch was taken

Modern day CPUs

- Deeply pipelined stages
- Complex Instructions
 - Vector instructions (SIMD)
 - AMX instructions to support AI applications
 - Variable length instructions (VLIW)
- Memory
 - Address space : VA vs PA
 - On-chip vs off-chip memory
 - Mem hierarchy (caches)
- Branch Prediction
- Out of order execution
- Multi cycle execution – FMAs, ALUs, Integer/Floating Point
- Multi core processors
- Multi threading

Memory Hierarchy



Intel Core i9-12900K, P-Core: Golden Cove. DDR5-5200 [export](#)

Modern Day CPU

ID/RF

- Instruction dispatches from IQ
- Multiple decoders
- Ucode instr -> uops
- Resource allocation (Register renaming/scoreboarding for out of order execution)

MEM + Mid level \$

- Loads/Stores handling
- Data TLB
- L1\$ → L2\$
- Prefetchers!!
- Sends requests (that miss all internal caches) to uncore, and tracks life of such requests

